Trajectory Optimisation: The Local Planner

Contents

1. Overview . . . 3
2. Global plan and local planner . . . 5
   - 2.1 Levels of planning . . . 5
   - 2.2 The global plan and the local planner in numbers . . . 6
   - 2.3 One optimisation, two loops . . . 7
3. The trajectory optimisation problem . . . 8
   - 3.1 What a trajectory is . . . 8
   - 3.2 Decision variables, cost and constraints . . . 10
   - 3.3 Hard and soft constraints . . . 12
4. Smoothing as optimisation: the elastic band . . . 14
   - 4.1 One corner by hand . . . 14
   - 4.2 Five points: the smoothing matrix . . . 16
   - 4.3 Smoothing the whole route . . . 18
5. The timed elastic band . . . 19
   - 5.1 Poses with time steps . . . 19
   - 5.2 The cost: time plus penalties . . . 21
   - 5.3 Solving it: sparse least squares . . . 22
6. Direct collocation . . . 24
   - 6.1 Functions into numbers: states at points . . . 25
   - 6.2 The trapezoid rule by hand . . . 25
   - 6.3 TEB and Apollo's path QP are collocation . . . 27
7. Local minima and warm starts . . . 28
8. The receding-horizon loop . . . 30
9. Refine once: one solve for a whole drive . . . 32
10. Real systems . . . 35
    - 10.1 Solvers, and path then speed . . . 35
    - 10.2 TEB in ROS and Nav2 . . . 36
    - 10.3 Autoware . . . 36
    - 10.4 Apollo . . . 37
    - 10.5 MoveIt . . . 38
11. Summary . . . 39
12. Sources . . . 40

13. Key terms 42

Where this fits:

- **Builds on:** A* search; Kinodynamic planning and phase space.
- **Leads to:** Path following and trajectory tracking; Direct and indirect methods; Single and multiple shooting.
- **Compare with:** Motion primitives and lattice search; Path smoothing; Single and multiple shooting.

1. Overview

Key point: A global planner makes a route from the map, rarely; a local planner turns the next few metres of it into a timed trajectory, within the robot's limits and around things the map does not have, by solving an optimisation problem every 0.1 s. When a person steps onto the route in front of the robot, the local planner bends round them, and the robot reaches the door at 8.0 s instead of 6.7 s.

The robot can already find its way to the door. A* on the room's map, cut into 0.5 m cells, gives the shortest route from its start cell to the goal cell under the door: 3.414 m. Planning with motion limits puts the speed into the robot's state, and path smoothing rounds a route's corners. Each of these runs once, on the map, before the robot moves.

The robot starts at rest at (2.0, 1.0) facing 30° and must stop at the door pose, (3.6, 3.5) facing 90°, the same goal as in planning with motion limits. The robot is a disk of radius 0.25 m; each wheel turns at most 0.6 m/s, and its forward speed changes by at most 0.5 m/s every second. At 2.4 s, with the robot at 0.595 m/s, a person steps onto the route and stands there: a disk of radius 0.2 m at (2.90, 2.75), 0.47 m in front of the robot's edge. The map does not have them. Four things go wrong (notebook):

- **The route runs through the person.** It passes 0.15 m from their centre, so the robot's disk would overlap them by 0.30 m.
- **The route has no clock.** It says where to drive, not how fast or how hard to turn at each moment within 0.6 m/s per wheel and 0.5 m/s^2.
- **The route has corners.** It turns 45° at three cell centres, and a robot at speed cannot turn on a point.
- **There is no time to plan the whole map again.** The robot covers 0.06 m every 0.1 s. Navigation stacks re-plan the whole map only every 2 s in one benchmark (Rösmann et al. 2021), or only when a new goal comes in (Autoware mission planner), because planning a long route again is often too slow for real time (Rösmann et al. 2013).

Figure 1 plays the drive this Note builds. Watch the orange band, the plan for the next 1.5 m that is solved again every 0.1 s, bend left round the person in the cycle they appear, and the blue robot follow it to the door. The drive comes from this Note's notebook: our own Python local planner, built the way TEB is (Section 5), not the ROS package itself.

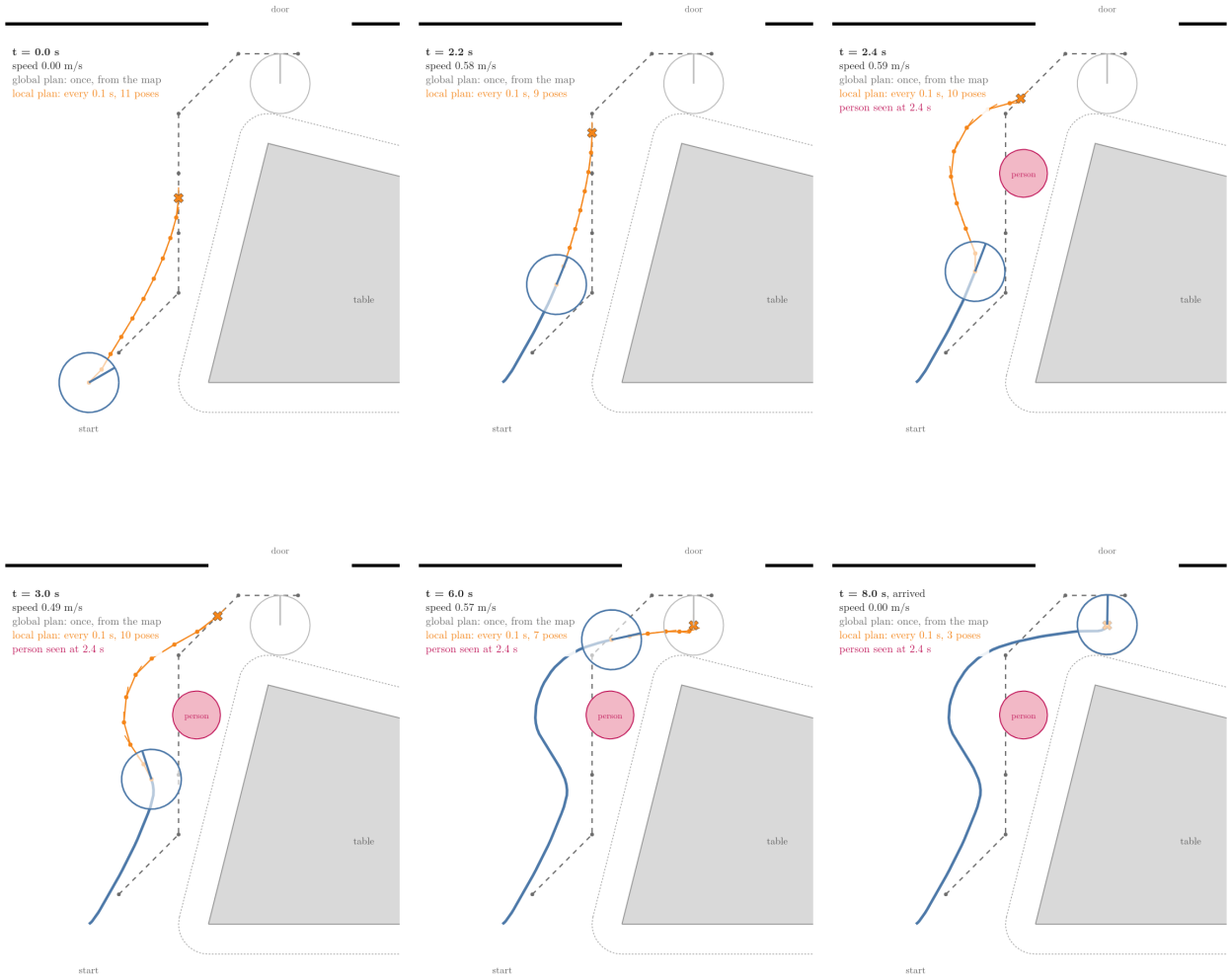


Figure 1: The robot (blue disk, with a line for its heading) drives from the start to the door pose (pale circle), one frame per 0.2 s. Grey dashed: the global plan, RO-034's A* route through cell centres, made once from the map. Orange: the local plan, the poses (dots, each with a short line for its heading) of a timed elastic band from the robot to the local goal (cross), 1.5 m further along the route; it is optimised again every 0.1 s. Blue line: where the robot has driven. At 2.4 s a person (pink) steps onto the route; in that cycle the band bends left round them. The last frame holds at the arrival, 8.0 s

The robot never stops: it slows to 0.44 m/s while it swerves, passes 0.14 m from the person, and reaches the door at 8.0 s, against 6.7 s when nobody steps in (notebook). Every 0.1 s the local planner solves one trajectory optimisation problem over the next 1.5 m, starting from its answer of the cycle before. This Note builds that, step by step:

- the split into a global plan and a local planner, with the rates and windows of real systems (Section 2);
- the optimisation problem: what is chosen, what it costs, what must hold (Section 3);

- smoothing a route as an optimisation problem, the elastic band (Section 4);
- the timed elastic band, the local planner of Figure 1 (Section 5);
- turning the problem into numbers, and direct collocation, the way TEB and Apollo do it (Section 6);
- why the first guess decides which side of the person the robot takes (Section 7);
- the loop that re-plans every 0.1 s (Section 8), and the other use, one solve for a whole drive (Section 9);
- how TEB, Autoware, Apollo and MoveIt use all this (Section 10).

2. Global plan and local planner

> **Key point:** Planning is split into levels: a global plan over the whole map, made rarely, and a local planner that plans the next few metres or seconds many times a second, with the time, the motion limits and the obstacles the map lacks. The local planner's trajectory comes from an optimisation that is either solved once from a search's path or solved again every cycle from its own last answer.

The opening's first problem is the person the map did not have: a route made from the map cannot know them.

2.1 Levels of planning

A car's planning is split into levels, each working coarser than the one below it, with route planning (G-2870), which picks the roads and lanes, at the top. Self-driving cars stack four levels (Paden et al. 2016 §II):

1. route planning picks the way through the map to the destination;
2. the behavioural layer picks the task for now: follow the lane, stop at the line, give way;
3. local motion planning turns that task into a path or a trajectory for the next stretch, around what the sensors see;
4. feedback control turns the trajectory into wheel or steering commands and corrects the robot as it drives.

A robot in a building splits its planning the same way (Figure 2). The planner that finds a route over the whole map, from the robot to the goal, and runs rarely is the **global planner** (G-2910); its route is the global plan. In the room it is A* on the 0.5 m grid, 3.414 m. The level that decides which task the robot carries out next, such as going on, stopping or giving way, is the **behavioural layer** (G-2912); in the room its task is simply "follow the route to the door". The planner that turns the next few metres or seconds of the global plan into a timed trajectory, within the robot's limits and around obstacles it sees, again and again while the robot drives, is the **local planner** (G-2911).

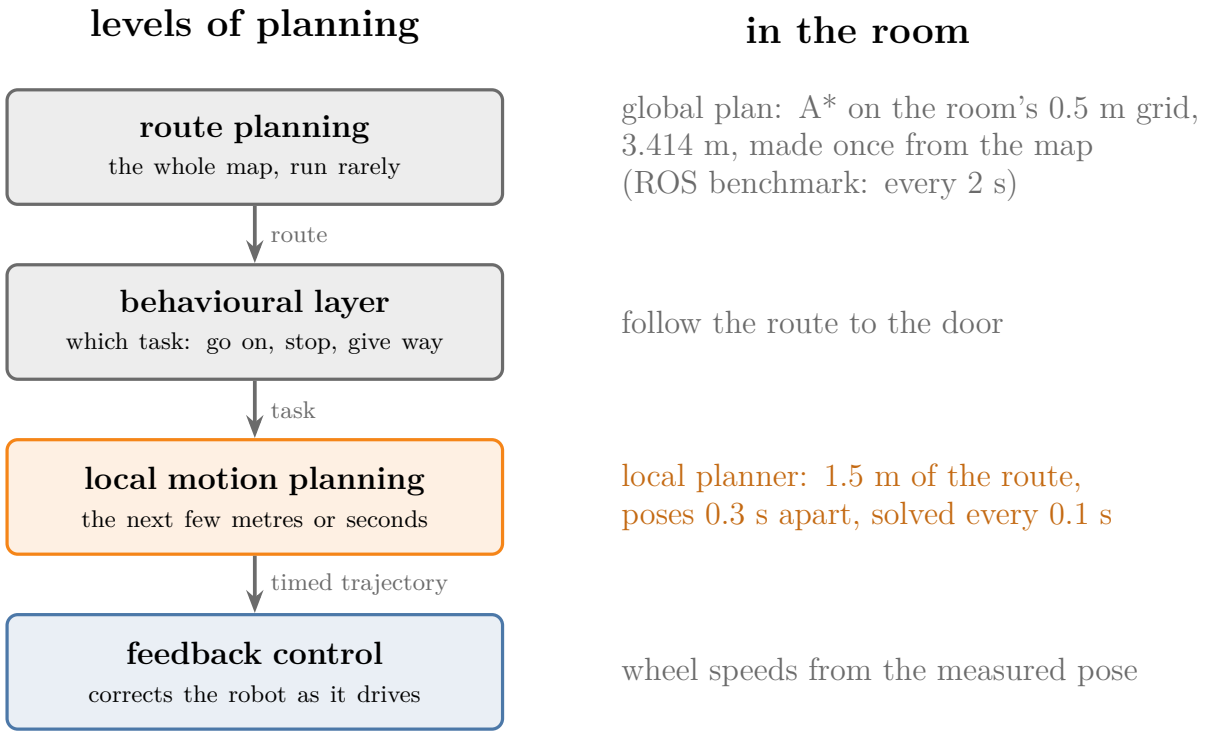


Figure 2: The four levels of planning (Paden et al. 2016, Fig. II.1), left, and what each does in the room, right. Route planning works on the whole map and runs rarely: here A* gives the 3.414 m global plan once (in a ROS benchmark the global plan is refreshed every 2 s). The behavioural layer picks the task: follow the route. Local motion planning, the local planner, plans 1.5 m of the route with poses 0.3 s apart and solves again every 0.1 s. Feedback control turns the plan into wheel speeds

2.2 The global plan and the local planner in numbers

The split shows in the numbers of real systems (Rösmann et al. 2021; teb_local_planner; autoware_launch; Apollo planning; Fan et al. 2018):

System	Global plan	Local window	Local rate
ROS benchmark with TEB	every 2 s	1.5 m of plan	10 Hz
TEB defaults	any planner	up to 3 m, poses 0.3 s apart	each control cycle (10 Hz in its tutorial)
Autoware	when the goal is given	path 100 m, speed 200 m	10 Hz
Apollo	on request	path 100 m, speed 7 s	each prediction message
our room	once	1.5 m, poses 0.3 s apart	10 Hz

The local planner adds three things the global plan lacks:

- **time and speed:** the elastic band, the method TEB grew from, shortens a path but takes no account of time or the robot's dynamics (Rösmann et al. 2017);
- **obstacles the map did not have:** the world "is subject to continuous change due to partial, incomplete maps and dynamic obstacles" (Rösmann et al. 2013);
- **the motion limits:** the global plan need not respect the robot's kinematics at all (Rösmann et al. 2017).

The local planner follows the global plan through one point: the pose on the global plan at the end of its window, facing along the plan there, which it must reach. That pose is the **local goal** (G-2929), TEB's "virtual goal" (TEB FAQ). In the room the window is 1.5 m of route, so at the start the local goal is (2.75, 2.54) facing 90°; once the end of the route is within 1.5 m, the local goal is the door pose itself. The local goal moves along the route with the robot, and the local planner keeps planning towards it.

2.3 One optimisation, two loops

The local planner of Figure 1 solves an optimisation problem; Section 3 states it. The same kind of problem is used in two loops, which differ in where the solver's first guess comes from (Figure 3):

- **refine once:** a search or sampling planner, such as A*, Hybrid A* or RRT, finds a path; the optimiser starts from that path and turns it into a smooth, timed trajectory once; a controller then follows it. Apollo parks this way (Hybrid A*, then smoothing) and MoveIt can plan arm motions this way (a sampling planner, then an arm optimiser such as CHOMP or STOMP) (Apollo planning; MoveIt 2 docs);
- **re-plan every cycle:** each cycle, the optimiser plans the next few seconds, starting from its own answer of the cycle before, moved to where the robot now is; the robot drives the first 0.1 s and the cycle repeats. TEB, Autoware's lane driving and Apollo's public-road planner work this way (Rösmann et al. 2017; autoware_launch; Apollo planning).

A first guess for a solver taken from an earlier answer, such as the last cycle's trajectory or a planner's path, is a **warm start** (G-2914): TEB starts each cycle from the trajectory it planned the cycle before (Rösmann et al. 2015; Rösmann et al. 2017). Planning over a window of fixed length that starts at the robot's current state, applying only the first part of the plan, then planning again from the new state, so the window slides forward with the robot, is planning on a **receding horizon** (G-2913) (Rösmann et al. 2021). The horizon is the end of the window: 1.5 m of route in the room.

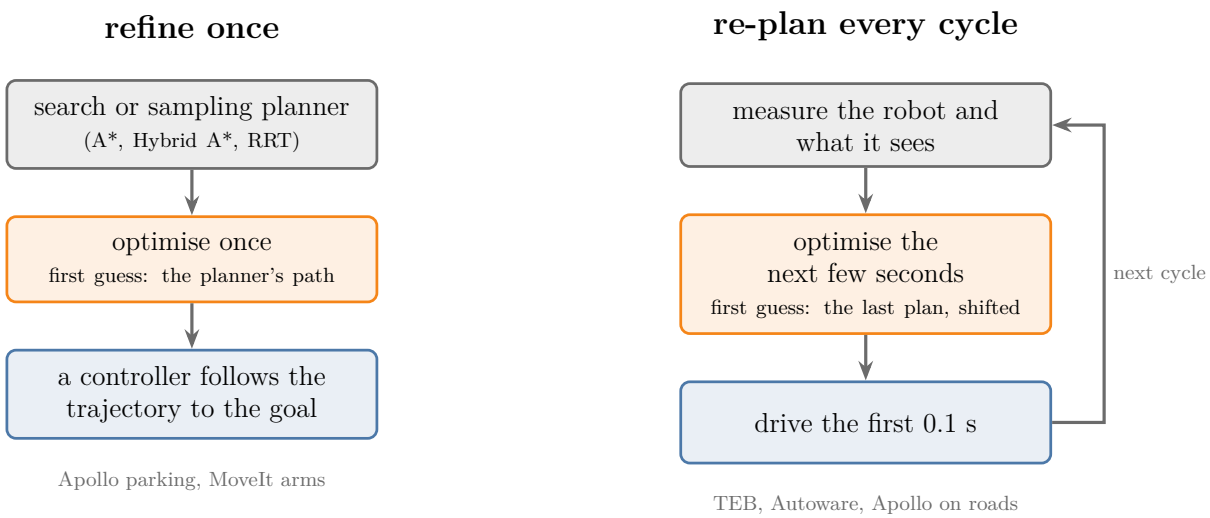


Figure 3: One optimisation, two loops. Left, refine once: a search or sampling planner finds a path; the optimiser starts from it and solves once; a controller follows the result to the goal (Apollo parking, MoveIt arms). Right, re-plan every cycle: measure the robot and what it sees; optimise the next few seconds, starting from the last plan shifted to the robot; drive the first 0.1 s; repeat (TEB, Autoware, Apollo on roads)

The room uses both: Section 8 runs the loop of Figure 1, and Section 9 refines the A* route once into a whole drive. Both need the problem written down first.

3. The trajectory optimisation problem

> **Key point:** A trajectory gives the robot's state at every moment and the controls that produce it. Trajectory optimisation chooses the numbers that describe it, the decision variables, to make a cost such as the time as small as possible while the motion rules, the start and the end, the limits and the clearance from obstacles hold, either exactly (hard constraints) or by paying a penalty (soft constraints).

The opening's second problem was the clock: the route says where to drive, not when. This section says what the local planner must find.

3.1 What a trajectory is

The idea. A path is a line on the floor. A trajectory is the same line with a clock on it: where the robot is at each moment, how fast it goes and how it turns. Figure 4 shows the local planner's answer in the cycle the person appears: each orange dot is a pose the robot should reach, labelled with the time from now, and the right panel gives the speed between neighbouring dots.

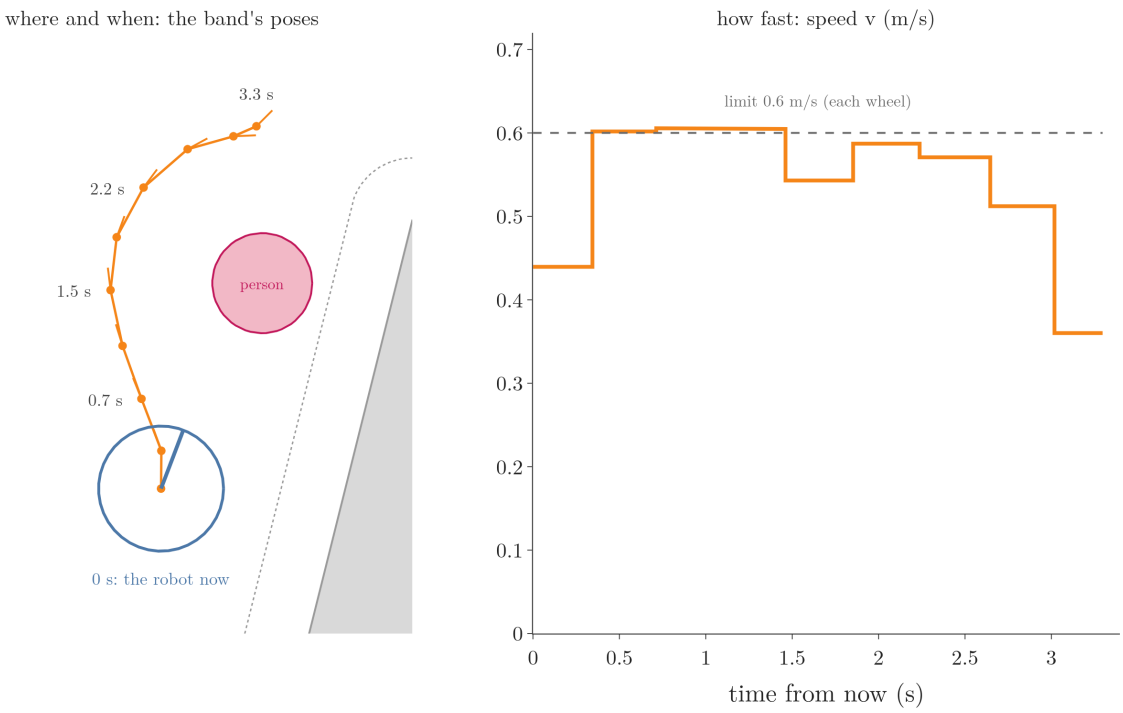


Figure 4: A trajectory: the local planner's band in the cycle the person is first seen, t = 2.4 s, after optimising. Left: its 10 poses (dots, with a short line for the heading), labelled with the time from now at which the robot should be there, from 0 s at the robot to 3.3 s at the local goal; it passes left of the person. Right: its speed between neighbouring poses against the time from now, with the 0.6 m/s wheel limit (dashed); the speed rises from 0.44 to about 0.6 m/s and falls to 0.36 m/s at the end of the window

To describe the robot at one moment we need four numbers, its **state** (G-2314), everything about the robot that matters for what happens next, and two commands, its **control inputs** (G-2298). With their values at the start:

- x, y, the position of the robot's centre, in m (2.0 and 1.0);
- θ, its heading, the direction it faces, in rad (30°, 0.524 rad);
- v, its forward speed, in m/s (0 at the start);
- a, the acceleration, how fast v changes, in m/s^2 (a control input);
- ω, the turn rate, how fast θ changes, in rad/s (a control input).

The rules that say how the state changes are the robot's kinematic model (G-2297), plus one line that makes the speed change only through the acceleration. A dot over a symbol means how fast that quantity changes, per second:

$$\dot{x} = v \cos\theta$$

$$\dot{y} = v \sin\theta$$

$$\dot{\theta} = \omega$$

$$\dot{v} = a$$

The speed is part of the state, not a command, because the robot cannot change its speed at once: at most 0.5 m/s every second (the acceleration limit).

The wheel limit. Each wheel turns at most 0.6 m/s. The wheel speeds for a forward speed v and turn rate ω are, with:

- v_R, v_L, the right and left wheel speeds, in m/s;
- L, the wheel separation, 0.2 m.

$$v_R = v + \omega L/2$$

$$v_L = v - \omega L/2$$

Both must stay at most 0.6 m/s. Driving at 0.5 m/s:

$$0.5 + \omega \times 0.1 \leq 0.6$$

$$\omega \leq 1 \text{ rad/s}$$

At rest the robot can spin at up to 6 rad/s. Turning costs speed: the faster the robot drives, the slower it can turn.

The standard term. A description of the robot's state at every moment of a time interval, together with the controls that produce it, is a **trajectory** (G-2886) (Kelly 2017; Lynch and Park §10.7).

3.2 Decision variables, cost and constraints

The trajectory of Figure 4 was chosen among many; choosing needs three things: what we may change, what makes one answer better than another, and what every answer must obey.

What we choose. A computer cannot choose a function of time with a value at every moment, so the trajectory is described by a list of numbers: Figure 4's band is 10 poses and the 9 time steps between them. The numbers an optimisation is free to choose are its **decision variables** (G-2915). Section 6.1 shows the ways to pick them.

What it costs. "Best" needs a number, the cost. The local planner of Figure 1 and the drive of Section 9 ask for the least time, the duration of the drive. Another common cost is

the least effort in a fixed time, a drive of exactly 8 s as gently as possible (Kelly 2017), which the next Note uses to compare solvers on the 8 s drive.

What must hold. The rules every acceptable answer must meet are its **constraints** (G-456):

1. the motion rules of Section 3.1, at every moment;
2. the start, where the robot is now, and the end, the local goal or the door pose;
3. the limits at every moment: both wheels at most 0.6 m/s, the acceleration between -0.5 and 0.5 m/s^2, the speed at least 0;
4. the clearance: the disk's centre at least 0.25 m from the table, the walls and the person.

Finding the decision variables that make the cost as small as possible while every constraint holds is **trajectory optimisation** (G-2887) (Kelly 2017; Lynch and Park §10.7). In symbols (Lynch and Park §10.7), with:

- $\mathbf{x}$, the state, the four numbers (x, y, θ, v);
- $\mathbf{u}$, the controls, the two numbers (a, ω);
- f, the four motion rules of Section 3.1, which give how fast each state number changes;
- T, the duration of the drive, in s, the cost.

$$\text{find } a(t), \omega(t), T$$

$$\text{minimising } T$$

$$\text{with } \dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$$

$$\mathbf{x}(0) = \text{start}$$

$$\mathbf{x}(T) = \text{goal}$$

$$\text{and the limits at every } t$$

One trajectory, not a rule for every state. The best possible answer would be a policy (G-1512), the best controls from every state the robot could be in, but even a coarse grid of our 4-number state holds over 10 million states. Trajectory optimisation finds one good trajectory from this start instead, which needs far less computation, and needs a new one when the robot drifts or a person steps in (Kelly 2017); the local planner solves again every 0.1 s.

3.3 Hard and soft constraints

The local planner must solve its problem in a few milliseconds, and a constraint can be handled in two ways that differ in how strictly it holds.

The idea. A wall is a hard limit: the robot cannot be on the other side, at any price. A speed limit with a fine is a soft one: going over is allowed, but it costs, and the more you go over, the more it costs. A solver can treat the 0.6 m/s wheel limit either way.

The standard terms. A constraint the answer must meet exactly, so that no answer that breaks it is accepted, is a **hard constraint** (G-2916). A constraint replaced by an extra cost, a penalty that is 0 while the constraint holds and grows with the amount it is broken, so that the problem has no constraints left, is a **soft constraint** (G-2917); handling constraints this way is the penalty method (Paden et al. 2016 §IV-C). TEB turns every constraint into a squared penalty, so its problem becomes a sum of squares (Rösmann et al. 2017).

On one piece of the band. Take one straight piece of a band, 0.18 m long, with both wheels at the speed v. Its time cost is the time step squared; its penalty for the wheel limit, with the weight 2 that TEB gives its speed limit, starts 0.05 m/s inside the limit, at 0.55 m/s, a safety margin (teb_local_planner). Figure 5 plots both against the speed.

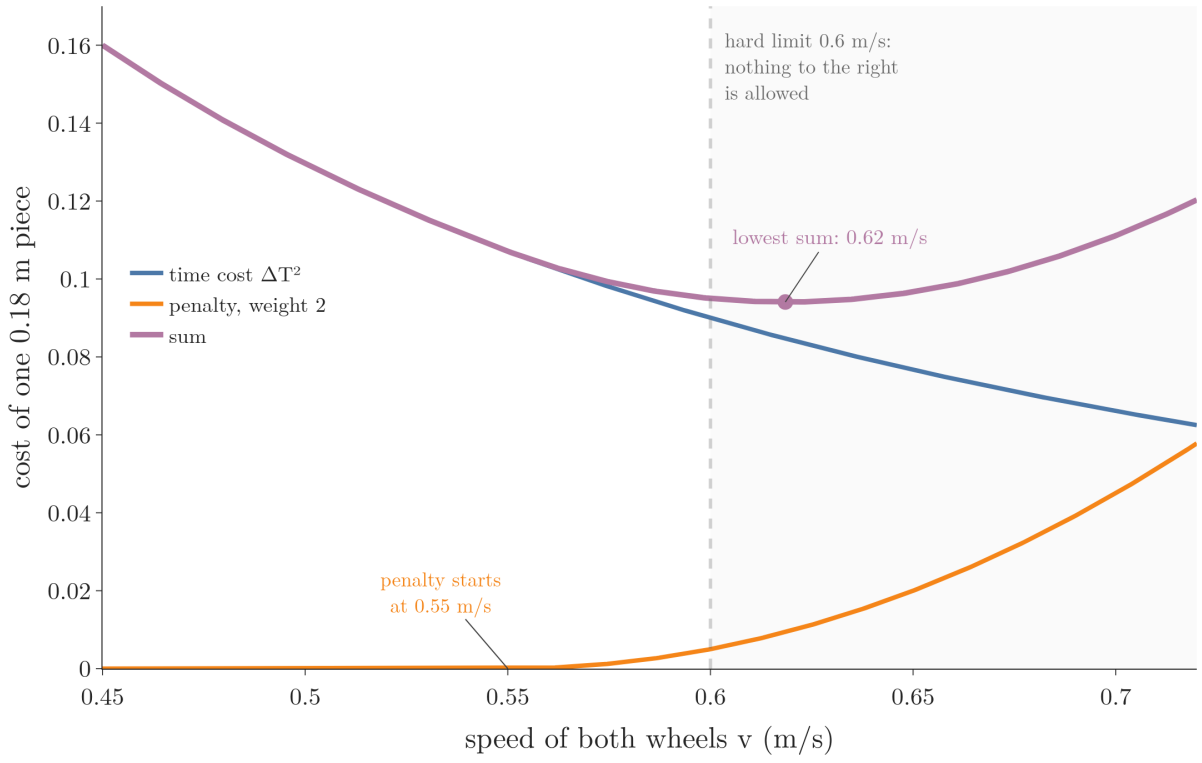


Figure 5: A hard limit against a soft one, on one straight piece of a band 0.18 m long, both wheels at the speed v. Blue: the time cost, the time step squared, which falls as the speed rises. Orange: the penalty for the wheel limit, with TEB's weight 2, zero below 0.55 m/s. Purple: their sum, lowest at 0.62 m/s, above the 0.6 m/s limit. Right of the dashed line, in grey, no speed is allowed under a hard limit

At 0.6 m/s, with:

- ℓ, the length of the piece, 0.18 m;
- v, the speed of both wheels, in m/s, here 0.6;
- ΔT, the time the piece takes, ℓ divided by v, in s;
- w, the penalty's weight, 2;
- $v_{\max}$, the wheel limit, 0.6 m/s;
- ϵ, the margin, 0.05 m/s;
- C, the cost of the piece, the time cost plus the penalty, in s^2.

$$\Delta T = 0.18/0.6 = 0.3 \text{ s}$$

$$\ell^2 = 0.18^2 = 0.0324$$

$$v^3 = 0.6^3 = 0.216$$

$$\text{time cost} = 0.3^2 = 0.09$$

$$\text{penalty} = 2 \times (0.6 - 0.55)^2 = 0.005$$

$$\text{slope of time cost} = -2 \times 0.0324/0.216$$

$$= -0.3$$

$$\text{slope of penalty} = 2 \times 2 \times 0.05 = 0.2$$

$$\text{slope of the sum} = -0.3 + 0.2 = -0.1$$

$$C = \ell^2/v^2 + w\,(v - v_{\max} + \epsilon)^2$$

$$\frac{dC}{dv} = -2\ell^2/v^3$$

$$+\ 2w\,(v - v_{\max} + \epsilon)$$

The sum still falls at 0.6 m/s, so its lowest point lies above the limit: at 0.62 m/s (notebook). With a weight of 1000 instead of 2 the lowest point moves to 0.550 m/s, inside the limit. A soft constraint holds only as well as its weight makes it: the answer of the penalised problem equals the constrained one only when every weight grows without end, and very large weights make the problem hard to solve accurately, so TEB's paper uses weight 1 for most penalties and about 1000 for the motion rule (Rösmann et al. 2017).

Penalties let a solver start from any guess, even one that breaks every limit, but the answer it settles on may still break some (Paden et al. 2016 §IV-C). So the robot keeps its own hard limits underneath: in the room, the wheels never pass 0.6 m/s and the speed never changes faster than 0.5 m/s^2, whatever the band asks for. The problem is stated; the first thing to optimise with it is the global plan's shape.

4. Smoothing as optimisation: the elastic band

> **Key point:** Smoothing a route can be written as one optimisation problem: move each point sideways, within a bound, to make the sum of squared second differences, how far each point lies from the middle of its neighbours, as small as possible. The cost is a quadratic program whose matrix has the same 1, 5, 6 pattern in Autoware's path smoother and in Apollo's reference-line smoother.

The opening's third problem was the route's corners: it turns 45° at (2.75, 1.75), (2.75, 3.25) and (3.25, 3.75), and a robot at speed cannot turn on a point.

Path smoothing (G-2842) pulled each point a little towards its neighbours, sweep after sweep. Real planners write the same wish as one cost, add bounds on how far each point may move, and hand the whole problem to a solver, so that every point moves at once to the best place the bounds allow (Autoware path smoother; Apollo planning).

4.1 One corner by hand

The idea. Three points are straight and evenly spaced when the middle one sits exactly halfway between the other two. How far it sits from that halfway point, doubled, measures how sharply the path bends there.

Take the first corner of the route and its two neighbours (Figure 6):

- p_0, p_1, p_2, three route points, in m: (2.25, 1.25), (2.75, 1.75) and (2.75, 2.25).

$$p_0 - 2p_1 + p_2$$

$$= (2.25 - 5.5 + 2.75,\ \ 1.25 - 3.5 + 2.25)$$

$$= (-0.5,\ \ 0)$$

Its squared length is 0.25 m^2. For three points on a straight line, evenly spaced, the result is (0, 0). A point's two neighbours minus twice the point is the **second difference** (G-2918): the change from one step to the next, so it is large where the path turns or its spacing changes. In symbols, with:

- p_k, point number k of the route, in m;
- Δ_k, the second difference at point k, in m.

$$\Delta_k = p_{k-1} - 2p_k + p_{k+1}$$

Sideways moves and a bound. Autoware's smoother lets each point move only sideways, at right angles to the path, and only by a small amount, 0.1 m on a lane, so that the smoothed path stays close to the planned one (Autoware path smoother). Here sideways means at right angles to the line through the two neighbours:

- d, the sideways move of p_1, in m;
- n, the sideways direction at p_1, a unit vector (G-2048), (−0.8944, 0.4472);
- Δ, the second difference before the move, (−0.5, 0);
- c, the smoothing cost, the squared length of the second difference after the move, in m^2.

Moving p_1 by d along n changes the second difference:

$$p_0 - 2\,(p_1 + d\,n) + p_2$$

$$= \Delta - 2d\,n$$

The dot product (G-634) of n and Δ:

$$n \cdot \Delta = (-0.8944)(-0.5) + 0.4472 \times 0$$

$$= 0.4472$$

$$c = 0.25 - 4 \times 0.4472\,d + 4d^2$$

$$= 0.25 - 1.789\,d + 4d^2$$

$$c(0.1) = 0.25 - 0.1789 + 0.04$$

$$= 0.111$$

The same steps in symbols:

$$c = \|\Delta - 2d\,n\|^2$$

$$= \|\Delta\|^2 - 4d\,(n \cdot \Delta) + 4d^2\|n\|^2$$

$$= \|\Delta\|^2 - 4d\,(n \cdot \Delta) + 4d^2$$

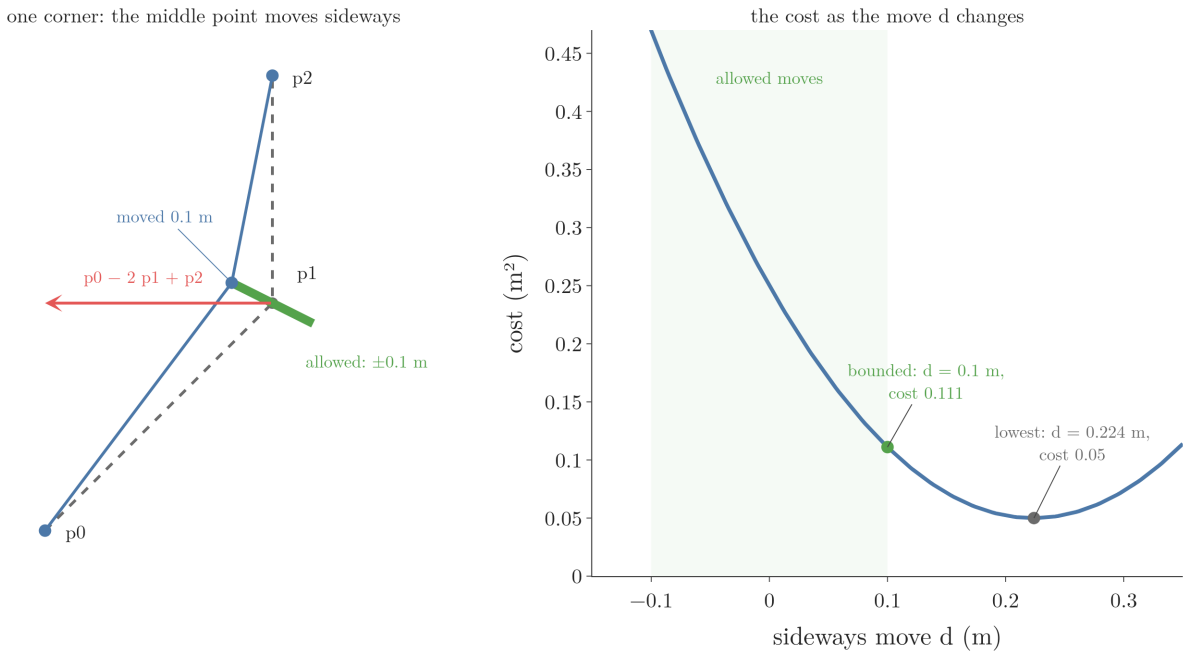


Figure 6: Smoothing one corner of the A* route by hand. Left: three route points p0, p1, p2 (grey dashed); the middle one may move only sideways, by at most 0.1 m each way (green bar); the red arrow is the second difference p0 − 2 p1 + p2 = (−0.5, 0); blue: p1 moved 0.1 m. Right: the smoothing cost against the sideways move d, 0.25 − 1.789 d + 4 d^2, lowest at d = 0.224 m (cost 0.05 m^2), but the bound (green band) stops the move at 0.1 m (cost 0.111 m^2)

The cost keeps falling until 0.224 m (Figure 6, right), but the bound stops the move at 0.1 m, where the cost is 0.111 m^2, down from 0.25 m^2.

4.2 Five points: the smoothing matrix

One corner moved alone; on a whole route every point is the middle of one second difference and a neighbour in two others, so the moves pull on each other. With five points in a row, numbered 1 to 5, the three second differences are rows of a matrix (G-1180) D, a table of numbers, acting on the five x coordinates (the same for y), with:

- $\mathbf{x}$, the five x coordinates as a column;
- D, the second-difference matrix, one row $(1, -2, 1)$ per middle point:

$$D = \begin{bmatrix} 1 & -2 & 1 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 1 & -2 & 1 \end{bmatrix}$$

The sum of the squared second differences, written with D:

$$\text{sum} = \|D\mathbf{x}\|^2$$

$$= \mathbf{x}^\mathsf{T}(D^\mathsf{T}D)\,\mathbf{x}$$

with:

- $D^\mathsf{T}D$, the matrix of the smoothing cost, D turned on its side (its transpose) times D:

$$D^\mathsf{T}D = \begin{bmatrix} 1 & -2 & 1 & 0 & 0 \\ -2 & 5 & -4 & 1 & 0 \\ 1 & -4 & 6 & -4 & 1 \\ 0 & 1 & -4 & 5 & -2 \\ 0 & 0 & 1 & -2 & 1 \end{bmatrix}$$

Why the diagonal reads 1, 5, 6, 5, 1: the middle point appears in three second differences, with the factors 1, -2 and 1, the second point in two, the end point in one:

$$\text{point } 3 = 1^2 + (-2)^2 + 1^2 = 6$$

$$\text{point } 2 = 1^2 + (-2)^2 = 5$$

$$\text{point } 1 = 1^2 = 1$$

Autoware's elastic-band smoother builds exactly this matrix for x and y, turns it into one for the sideways moves, adds a small pull back to the original points (weight 0.001 against 1), and bounds every move; Apollo's reference-line smoother builds the same 1, 5, 6 pattern, with box bounds on each point and extra terms for length and for staying near the original line (Autoware path smoother; Apollo planning). A squared cost with linear bounds is a **quadratic program** (G-1598), for which a solver finds the exact best answer. Treating a path as a chain of points that the cost pulls straight, like a stretched rubber band, while bounds or obstacles hold it in place, and optimising it, is an **elastic band** (G-2919) (Rösmann et al. 2013; Autoware path smoother).

4.3 Smoothing the whole route

The route has 7 points; the two ends stay fixed and the 5 others may each move sideways by at most 0.1 m. Figure 7 solves it step by step: each step moves every point downhill on the cost and then clips it back inside its bound, a projected gradient step. Watch the corners pull in until three points sit at the end of their bars.

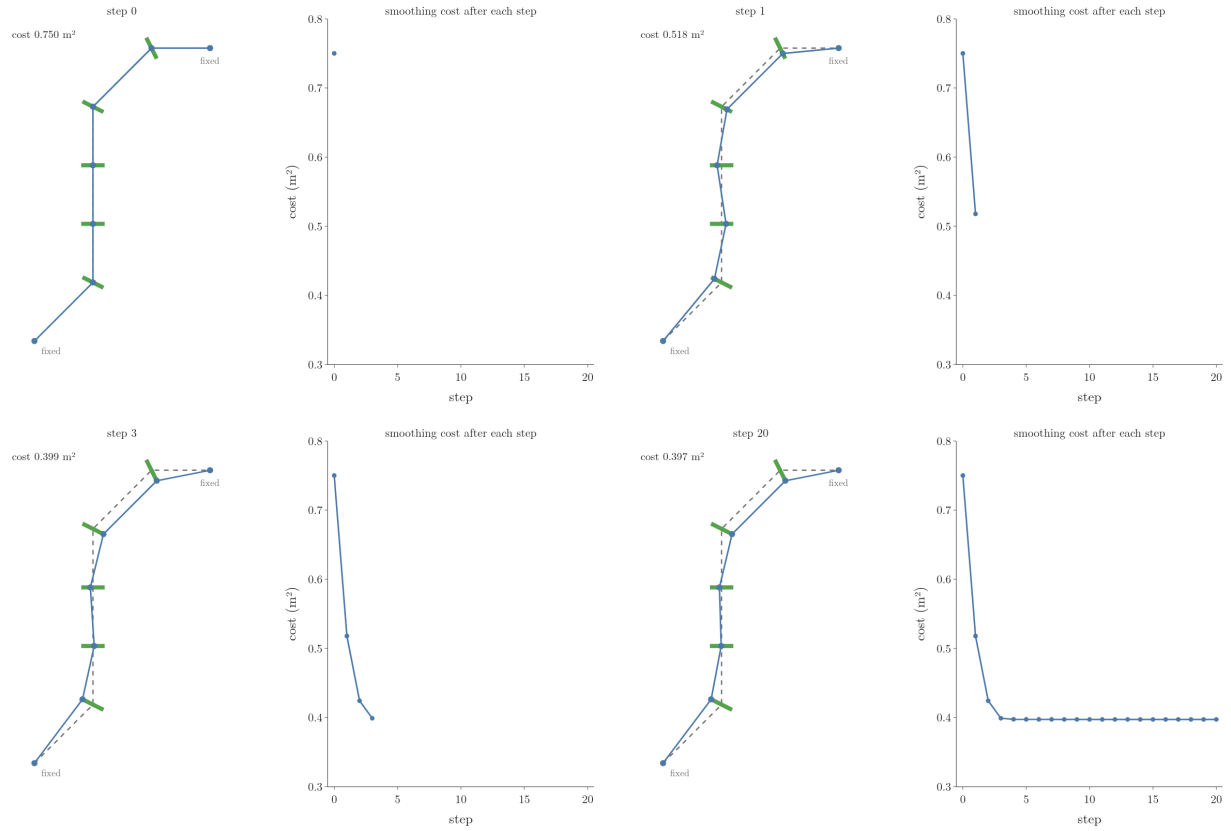


Figure 7: Smoothing the whole A* route as one optimisation, one solver step per frame. Grey dashed: the 7 route points, the global plan; the two ends stay fixed; green bars: the 0.1 m each way that each of the 5 others may move sideways. Blue: the points after the step. Right: the smoothing cost after each step, from 0.75 m^2 to 0.397 m^2; three points end at the end of their bars

The cost falls from 0.75 to 0.397 m^2 in about four steps; the three corner points move the full 0.1 m and the two on the straight piece barely move (notebook). The bound decides the answer here: the corners would like to move further. Autoware runs its smoother on 100 points 1 m apart at every new input, with a solver for quadratic programs, OSQP (Autoware path smoother). The smoother does no collision check: its job is a smooth reference for the next step (Autoware path smoother). The smoothed route still has no clock, knows nothing of the person, and sits inside 0.1 m of a route that runs through them; a band that also carries time can fix all three.

5. The timed elastic band

> **Key point:** A timed elastic band (TEB) is a list of poses with the time between neighbours. Speeds, turn rates and accelerations come from differences between neighbours; the cost is the sum of the squared time steps plus squared penalties for every broken rule, and Levenberg–Marquardt steps solve it fast because each term involves only a few neighbouring numbers.

The smoothed route of Section 4 still has no clock and runs through the person; the local planner of Figure 1 fixes both in one solve.

TEB is a local planner that plugs into the ROS navigation stack, and has been ported to Nav2 (teb_local_planner). The stack's own classic local planner, the dynamic window approach (DWA), tries a few constant pairs of speed and turn rate (by default 3 speeds and 20 turn rates, each held for 1.7 s), scores where each would take the robot, and keeps the best; it searches among sampled commands, while TEB adjusts every pose of its trajectory with a numerical solver (DWAPlanner.cfg; Rösmann et al. 2021). Classical local planning, a later Note (RO-078), teaches DWA.

5.1 Poses with time steps

The idea. Lay beads along the next 1.5 m of the route, the first on the robot and the last on the local goal, the route's pose 1.5 m ahead. Between each pair of neighbours, write how many seconds the robot takes. Now the beads say where, and the numbers between them say when: Figure 4 was such a band.

The standard term. A list of poses with the time between neighbouring poses as decision variables, optimised for the least total time within the robot's limits and clear of obstacles, is a **timed elastic band** (G-2920), TEB (Rösmann et al. 2013; Rösmann et al. 2017). With:

- x_k, y_k, θ_k, the position (in m) and heading (in rad) of pose number k, for k from 1 to N;
- ΔT_k, the time from pose k to pose $k+1$, in s;
- N, the number of poses, 11 in the first cycle.

The first pose is the robot and the last the local goal; both stay fixed while the others move (Rösmann et al. 2017). TEB keeps neighbours about 0.3 s apart. Before each round of solving it (teb_local_planner):

- splits a time step longer than 0.6 s in two;
- trims a step between 0.4 and 0.6 s back to 0.3 s and adds the rest to the next step;
- removes a pose where a step has shrunk below 0.2 s.

Speeds from differences. The band has no speeds among its numbers; they come from neighbours. In the first cycle the robot is at rest at (2.0, 1.0) facing 30°, and after optimising the next pose is (2.1054, 1.1087) facing 61.73°, 0.4831 s later (notebook). With:

- ℓ_k, the distance from pose k to pose $k+1$, in m, here 0.1514;
- v_k, the speed on that piece, in m/s;

- ω_k, the turn rate on that piece, in rad/s;
- v_0, the robot's speed now, 0 m/s;
- a_k, the acceleration from piece k to piece $k+1$, in m/s^2; a_1 is from the robot's speed now to the first piece.

$$v_1 = 0.1514/0.4831 = 0.3134 \text{ m/s}$$

$$\omega_1 = (1.0774 - 0.5236)/0.4831$$

$$= 1.1464 \text{ rad/s}$$

$$a_1 = (0.3134 - 0)/0.4831$$

$$= 0.649 \text{ m/s}^2$$

$$v_k = \ell_k/\Delta T_k$$

$$\omega_k = (\theta_{k+1} - \theta_k)/\Delta T_k$$

$$a_k = \frac{2\,(v_{k+1} - v_k)}{\Delta T_k + \Delta T_{k+1}}$$

The speed is the distance over the time, as for any trip. An acceleration compares two speeds, each the average over its own piece, so they belong to the middles of their pieces, which lie half the sum of the two time steps apart; dividing by half that sum gives the 2 in the last line (Rösmann et al. 2017). The first acceleration compares the first speed with the robot's speed now, over the first time step (teb_local_planner).

The motion rule. A wheeled robot cannot slide sideways, a nonholonomic constraint (G-2302), so between two poses it drives an arc of a circle. On a circular arc the line from one end to the other points exactly halfway between the headings at its two ends, so both headings make the same angle with it. TEB writes this as a number that must be 0 (Rösmann et al. 2013; Rösmann et al. 2017), with:

- Δx_k, Δy_k, the change in x and y from pose k to pose $k+1$, in m;
- e_k, the arc rule's miss, 0 when the two poses lie on one arc, in m.

$$e_k = (\cos\theta_k + \cos\theta_{k+1})\,\Delta y_k$$

$$-\,(\sin\theta_k + \sin\theta_{k+1})\,\Delta x_k$$

A check on the first cycle's first guess, before optimising: the first two poses face 30° and 37.5°, and the line between them points at 45°, not halfway at 33.75°; each coordinate changes by 0.1604 m:

$$e_1 = (0.8660 + 0.7934) \times 0.1604$$

$$-\,(0.5 + 0.6088) \times 0.1604$$

$$= 0.0883 \text{ m}$$

After optimising, the line points at 45.87°, halfway between 30° and 61.73°, and every e_k is below 0.0001 m (notebook).

5.2 The cost: time plus penalties

The band must be fast, obey the arc rule, and keep every limit and the clearance. TEB adds all of these into one sum of squares (Rösmann et al. 2017; teb_local_planner), with the weights of its default settings:

- the time: each time step squared, weight 1; the sum of squared time steps tends to make the steps equal (Rösmann et al. 2017);
- the arc rule: each e_k squared, weight 1000;
- the wheels: each wheel's excess over 0.55 m/s squared, weight 2 (our robot's wheel limit in place of TEB's separate speed and turn-rate limits);
- the acceleration: the excess of a_k over 0.45 m/s^2 squared, weight 1;
- the direction: how far a piece runs backwards, squared, weight 1;
- the clearance: at every pose and halfway between neighbours, how far the disk's edge comes within 0.15 m of an obstacle, squared, weight 50.

The clearance is TEB's `min_obstacle_dist` plus its margin; we set the distance to 0.1 m for the small room (TEB's default is 0.5 m). TEB checks the clearance only at its poses, and its FAQ warns that poses can spread apart across a gap (TEB FAQ); our band also checks the point halfway between neighbours.

In the first cycle the cost of the first guess, the route's poses spaced at full speed, is 21.38; after optimising it is 1.182 (notebook). The optimised band still asks for 0.649 m/s^2 at the start, above the 0.5 m/s^2 limit: the acceleration is a soft constraint of weight 1, cheaper to break a little than to lose time (Section 3.3). The robot's own limit holds it to 0.5 m/s^2, and the next cycle plans again from where the robot really is.

5.3 Solving it: sparse least squares

The cost is a sum of squares of numbers that each depend on a few neighbouring poses and time steps. Minimising such a sum is a least-squares problem, the kind ordinary least squares (G-1406) solves for a line in one formula, here with curved terms that need steps.

The steps. At the current band, write each squared number as a straight-line guess in the decision variables, using its Jacobian (G-980), the table of how each number changes with each variable. The sum of the squared guesses is a bowl whose lowest point can be solved for exactly: that jump is the Gauss–Newton step. Far from the answer the straight-line guesses are poor, so the jump is shortened by blending it with a small step straight downhill, more when a step fails to lower the cost and less when it succeeds. Gauss–Newton steps blended this way are the **Levenberg–Marquardt method** (G-2921) (Nocedal and Wright §10.3). TEB runs it in the graph-optimisation library g2o: 4 passes, each of which first resizes the band and then takes 5 Levenberg–Marquardt steps, with the clearance weight doubled after each pass (Rösmann et al. 2013; teb_local_planner).

Figure 8 runs these 4 passes in the cycle the person appears. Watch the band, which first runs through the person, slide left in the second pass, when the clearance weight has doubled.

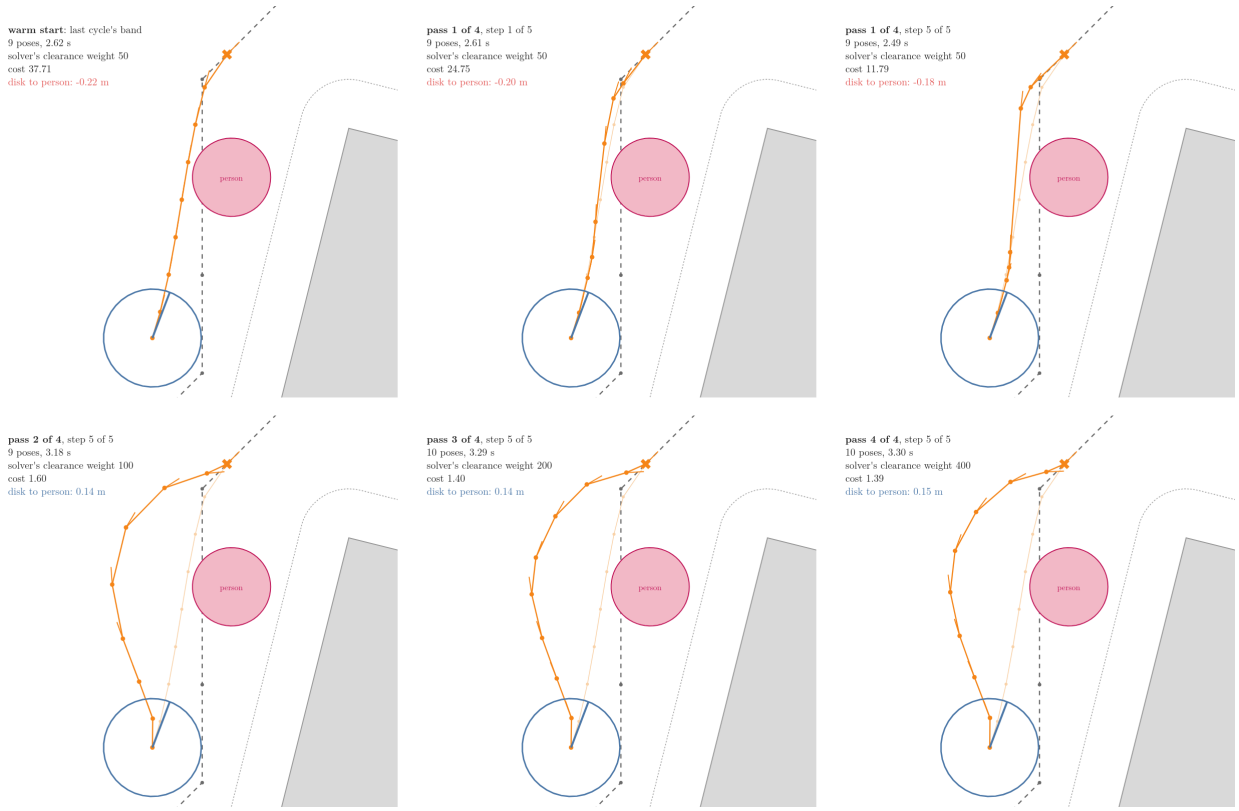


Figure 8: The timed elastic band optimised, one step per frame, in the cycle the person is first seen (t = 2.4 s). First frame: the warm start, last cycle's band with its first pose moved to the robot and its last to the local goal (cross); it runs through the person, and the robot's disk would overlap them by 0.22 m. Then 4 passes: each resizes the band so neighbours stay about 0.3 s apart, then takes 5 Levenberg–Marquardt steps; the solver's clearance weight starts at 50 and doubles after each pass. The text gives the cost with TEB's set weights and the gap from the robot's disk to the person (negative: overlap). Last frame: 10 poses over 3.30 s, 0.15 m clear, cost 1.39

The band starts with 9 poses and an overlap of 0.22 m, cost 37.71, and ends with 10 poses over 3.30 s, 0.146 m from the person, cost 1.39 (notebook). Each resize changes the band a little, so the cost jumps at the start of each pass and the steps bring it down again.

Why it is fast. The number that ties the arc rule of poses 3 and 4 never involves pose 8. So in the matrix that each Levenberg–Marquardt step solves, most entries are 0: the matrix is **sparse** (G-1846), mostly zeros. Figure 9 marks which entries can be non-zero.

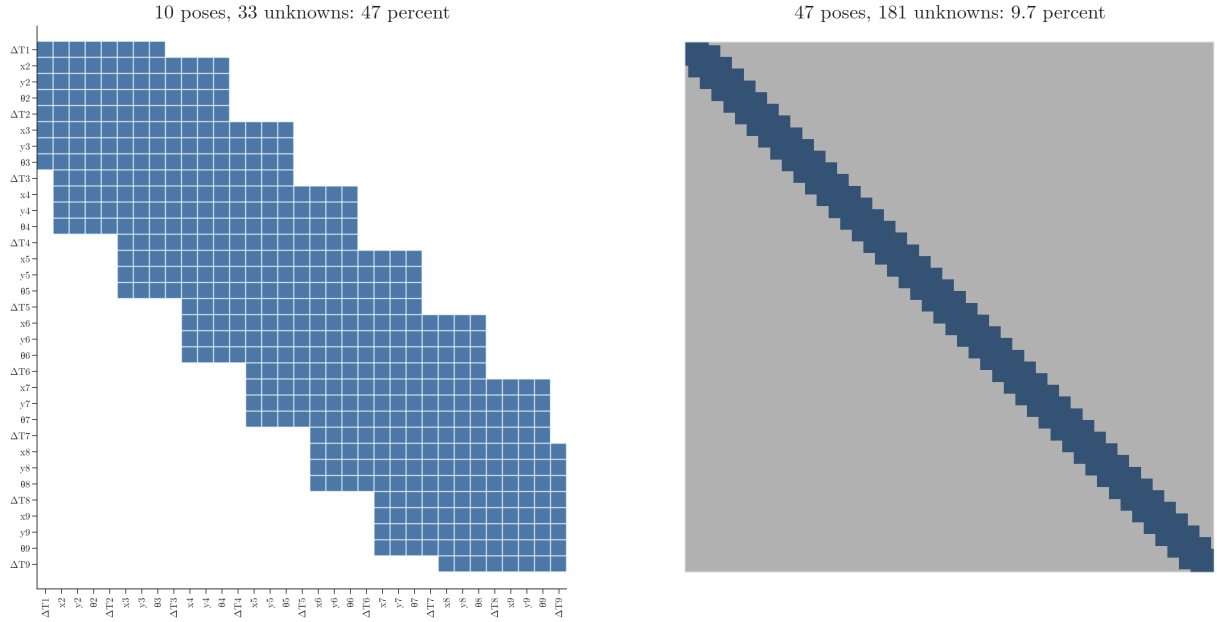


Figure 9: Which entries of the matrix each Levenberg–Marquardt step solves can be non-zero (dark), one row and one column per decision variable in TEB's order: time step 1, then x, y and heading of pose 2, time step 2, and so on. Left: the band of 10 poses from the cycle the person appears, 33 unknowns, 47 percent. Right: a band of 47 poses, 181 unknowns, 9.7 percent. The dark entries hug the diagonal, because every term involves only neighbouring poses and time steps

Our 10-pose band fills 47 percent of its matrix, a band of 47 poses only 9.7 percent (notebook); TEB's own 47-pose example filled 15 percent (Rösmann et al. 2013). Solvers for sparse matrices skip the zeros: in TEB's tests the run time grew in step with the number of poses, and one whole cycle of its solver took 2.1 ms on average (Rösmann et al. 2013). TEB's choice of decision variables, poses and time steps with the motion rule as a difference between neighbours, is one of three ways to turn a trajectory into numbers; the next section lays them out.

6. Direct collocation

> **Key point:** Turning a trajectory into a list of numbers is transcription. Direct collocation keeps the states and the controls at a row of points in time as unknowns and requires between neighbours that the change in state equals the trapezoid-rule area under its rate of change, so nothing is simulated and limits are simple; TEB and Apollo's path optimiser are both of this kind.

TEB's band of Section 5 turned the trajectory into poses and time steps, with the motion rule as a difference between neighbours. The refine-once drive of Section 9 needs the same step for any trajectory: a list of numbers that a solver can change.

6.1 Functions into numbers: states at points

The idea. A film is a list of still pictures, 24 every second, and the eye fills in between. A trajectory can be described the same way: keep the robot's state and controls at a row of moments, and fill in between. Figure 4's band is such a list, 10 poses of 3 numbers each and the 9 time steps between them:

$$10 \times 3 + 9 = 39 \text{ numbers}$$

The standard term. Turning a trajectory optimisation problem, whose unknowns are functions of time, into one whose unknowns are a finite list of numbers is **transcription** (G-2888) (Kelly 2017). There are three ways to pick the numbers (Lynch and Park §10.7):

1. **the path only**, and work out the controls from it; this fails for our robot, which has 2 controls for its 3 position numbers x, y, θ, so most paths cannot be driven (Lynch and Park §10.7);
2. **the controls only**, and find the states by simulating the motion rules: shooting, which the next Note (RO-231) teaches;
3. **the controls and the states**, and make the motion rules hold between them as equations: collocation, the choice TEB makes.

Points on the drive. Mark 31 points in time across the drive from the start to the door pose. At each point the state (4 numbers) and the controls (2 numbers) are unknowns, and for the least-time drive of Section 9 the duration is one more:

$$31 \times (4 + 2) + 1 = 187 \text{ numbers}$$

Between neighbouring points the motion rules must hold approximately: the change in each state must equal how fast it was changing, averaged over the two points, times the time between them. The solver moves all the points at once until every one of those equations holds (Kelly 2017).

The standard terms. Transcription in which the states and the controls at a set of points are all unknowns and the motion rules are constraints between them is **direct collocation** (G-2893) (Kelly 2017; Tedrake, Underactuated ch. 10). The points are the **knot points** (G-2895), also called collocation points, and each equation that ties neighbouring knot points together through the motion rules is a **collocation constraint** (G-2894) (Kelly 2017).

6.2 The trapezoid rule by hand

Take the straight drive to the door, 3.0 m along a line from rest to rest in the least time, the simplest drive, where the rule can be checked by hand. Its collocation with 10 intervals has a first interval of 0.6364 s, in which the speed rises in a straight line from 0 to 0.3182 m/s at 0.5 m/s^2 (notebook). The distance covered is the area under the speed, a trapezoid, here with one side 0, so a triangle (Figure 10):

$$\text{distance} = 0.6364/2 \times (0 + 0.3182)$$

$$= 0.1013 \text{ m}$$

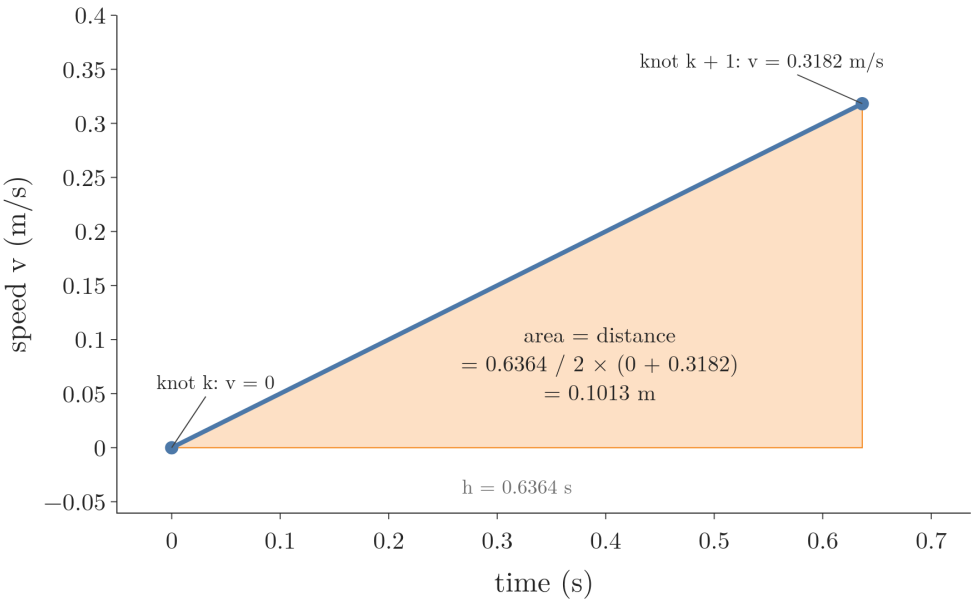


Figure 10: The trapezoid rule on the first interval of the straight drive, 0.6364 s long (10 intervals of the least-time collocation). The speed rises in a straight line from 0 at knot k to 0.3182 m/s at knot k + 1; the distance covered is the area under it (orange), 0.6364 / 2 × (0 + 0.3182) = 0.1013 m, exactly what speeding up at 0.5 m/s^2 for 0.6364 s gives

Constant acceleration gives the same distance, so here the rule is exact, because the speed changes in a straight line:

$$0.5 \times 0.5 \times 0.6364^2 = 0.1013 \text{ m}$$

The same rule for every state of the robot, with:

- $\mathbf{x}_k$, the state at knot point k (four numbers);
- $\mathbf{u}_k$, the controls at knot point k (two numbers);
- h, the time between knot points, in s (0.206 s for the least-time drive of Section 9);
- f_k, the rates of change at knot point k, the motion rules of Section 3.1 at $\mathbf{x}_k$ and $\mathbf{u}_k$.

$$\mathbf{x}_{k+1} - \mathbf{x}_k = \frac{h}{2}\left(f_k + f_{k+1}\right)$$

The equation is the collocation constraint of trapezoidal collocation (Kelly 2017, eq. 3.2). The rule assumes the rates change in a straight line between knot points; where the true rates

bend, it is only approximate (Kelly 2017), and more knot points shrink the error (collocation checked on the straight drive).

6.3 TEB and Apollo's path QP are collocation

Two real planners write their motion rules the same way, as equations between neighbouring points.

TEB. TEB's unknowns are the poses and the time steps at its points, and its motion rules are differences between neighbours: the speed is the distance over the time step, and the arc rule ties each pair of poses (Section 5.1). Collocation with the motion rules written as differences between neighbouring points is **finite-difference collocation** (G-2923); TEB is derived this way from a least-time control problem, and its penalties then turn the constrained problem into the least-squares problem of Section 5 (Rösmann et al. 2021).

Apollo's path. Apollo plans a car's path on a road as a sideways offset from the lane's centre line: at stations every 0.5 m along the line, for 100 m, it keeps the offset l, its slope l' and its curvature-like second slope l'' as unknowns, and lets the third slope, the jerk of the offset, be constant between neighbours (Zhang et al. 2020; Apollo planning). Then l'' changes in a straight line between stations, and the slope changes by the trapezoid area under it, exactly as the speed did under the acceleration in Section 6.2. With an example on 0.5 m stations:

- Δs, the distance between stations, 0.5 m;
- l''_i, l''_{i+1}, the second slope at two neighbouring stations, in 1/m, here 0 and 0.04;
- l'_i, l'_{i+1}, the slope at the two stations, with no unit.

$$l'_{i+1} - l'_i = 0.5/2 \times (0 + 0.04)$$

$$= 0.01$$

$$l'_{i+1} - l'_i = \frac{\Delta s}{2}\,(l''_i + l''_{i+1})$$

The equation is the collocation constraint of Section 6.2 again, exact because the jerk is constant (Zhang et al. 2020). Each unknown is the rate of change of the one before, a chain of integrators: jerk drives l'', which drives l', which drives l; solving for all of them at stations, with these equations between neighbours, inside bounds left and right, as a quadratic program (a squared cost with linear bounds), is **piecewise-jerk optimisation** (G-2924) (Zhang et al. 2020). Apollo uses the same idea for the speed along the path (Apollo planning).

Collocation's solver, like the band's, improves a first guess by small local steps, so where the first guess lies decides which answer it finds.

7. Local minima and warm starts

> **Key point:** An optimiser only goes downhill from where it starts, so it finds the lowest point of the valley it starts in, a local minimum, and a band that starts on the wrong side of an obstacle stays there. A search, which explores every way round, gives a first guess on the right side; the optimiser then makes it smooth and fast.

The opening's person stood right of the route's line, so the band slid left. Had they stood further left, the same steps would have pushed the band the other way.

The idea. Each Levenberg–Marquardt step of Section 5.3 moves the band a little downhill on the cost. A pose inside the person is pushed out of them by the shortest way. If the band runs left of the person's centre, the shortest way is left; if right, right. Once the band is on one side, every small step keeps it there: moving it to the other side would mean passing through the person, where the cost is high.

The person 0.25 m further left. We put the same person at (2.65, 2.75), 0.25 m further left, and solve the same cycle from two first guesses (Figure 11):

- **last cycle's band**, the warm start of Section 5.3, which runs just right of the person's centre;
- **a search:** A*, the search that made the global plan, on a local map of 0.1 m cells, with a cell blocked where the robot's disk centred there would overlap an obstacle, from the robot to the local goal; it expands 78 cells and finds a 2.107 m route left of the person (notebook).

Watch the left band get pushed right, into the gap between the person and the table, while the right one passes on the left.

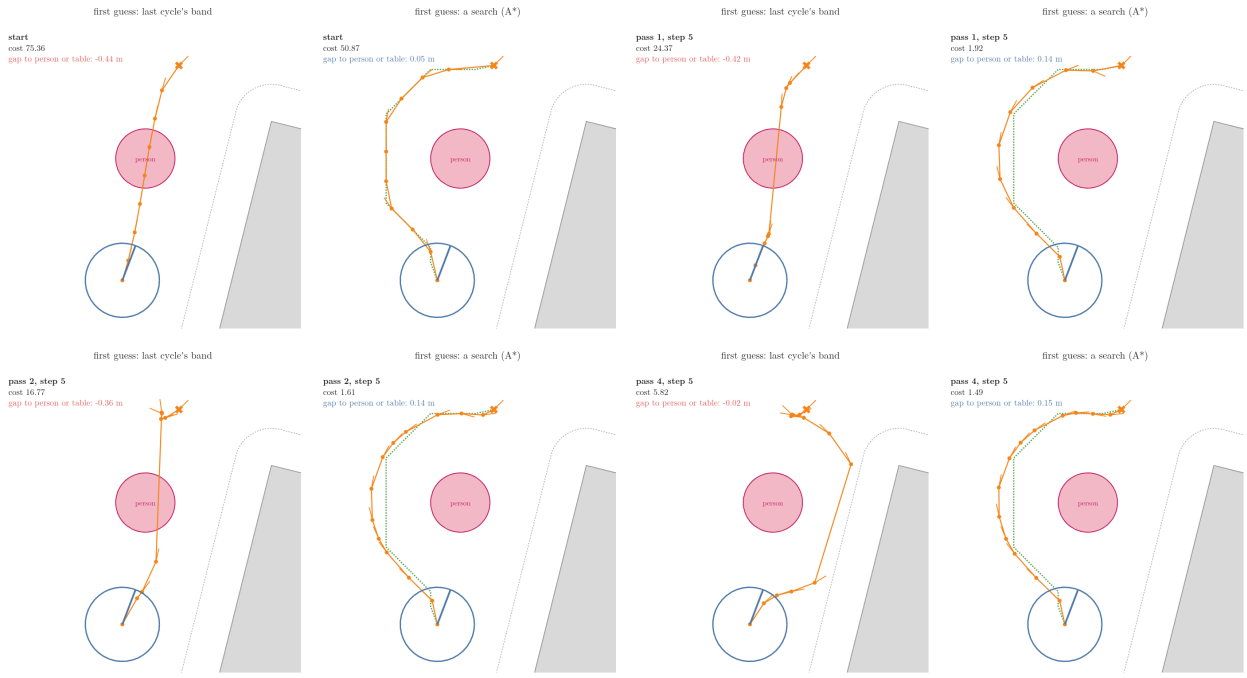


Figure 11: The same optimiser from two first guesses, one step per frame, with the person 0.25 m further left than in the Overview, at (2.65, 2.75). Left: started from last cycle's band, which runs just right of the person's centre; the steps push it right into the gap between the person and the table, where it settles with the robot's disk overlapping them by 0.02 m, cost 5.82. Right: started from the route of A* on a local map of 0.1 m cells (green dotted), which goes left of the person; the same steps give a band 0.15 m clear, cost 1.49. Both runs: TEB's 4 passes of 5 Levenberg–Marquardt steps; the cost is counted with TEB's set weights

The warm start settles on the right, where the gap between the person and the table is 0.564 m and the robot is 0.5 m wide: the band scrapes both, overlapping by 0.02 m, at a cost of 5.82. The search start settles on the left, 0.148 m clear, at a cost of 1.49 (notebook). Both are lowest points of their own valleys: a **local minimum** (G-1110) is lower than everything around it but not the lowest overall. Here the warm start found the worse one.

Why the two methods need each other. Optimisation methods converge fast to a locally best answer, but they cannot find the best one overall unless they are given a suitable first guess; graph searches do not get stuck in local minima, but they choose only among the paths their graph contains; so a coarse search is used to give the optimiser its first guess (Paden et al. 2016 §IV-A). A search makes the discrete choice, which side of the person, and the optimiser makes the answer smooth and fast. TEB's authors found the same: the band cannot cross to the other side of an obstacle, and which local optimum it reaches depends on how it starts (Rösmann et al. 2015; Rösmann et al. 2017). Real systems pair the two:

- TEB, by default, also optimises up to 5 bands that start on different sides of the obstacles, found by a search, and keeps the cheapest (teb_local_planner; Rösmann et al. 2015);

- Apollo's EM planner first searches for a rough answer by dynamic programming, then smooths it with a quadratic program (Fan et al. 2018);
- MoveIt can seed CHOMP with a sampling planner's path; its guide suggests this because CHOMP "can fail if it gets stuck in local minima due to a bad initial guess" (MoveIt 2 docs).

Two paths from the same start to the same goal that can be bent into each other without crossing an obstacle belong to the same **homotopy class** (G-2922); passing the person on the left and on the right are two classes, and an optimiser never leaves the class it starts in (Rösmann et al. 2015). Classical local planning, a later Note (RO-078), teaches how TEB finds and compares them. In the opening's room the band ran left of the person's centre, so the warm start was enough; the next section runs that loop.

8. The receding-horizon loop

> **Key point:** Every 0.1 s the local planner measures the robot, shifts its last band to the robot and the new local goal, optimises it, and drives its first 0.1 s; the window slides forward with the robot. Each solve starts from the last answer, so it only has to correct it, and the person enters the plan in the cycle they appear.

The opening's last problem was time: the person appeared 0.47 m in front of the robot, and the plan had to change within one cycle.

The idea. A driver does not plan the whole trip at once and then shut their eyes: they look a few seconds ahead, steer, and look again. The local planner does the same, every 0.1 s, on a receding horizon (Section 2.3). Each cycle has four steps (Figure 12):

1. **measure** where the robot is and how fast it goes; it has driven 0.1 s along the last band;
2. **warm start:** drop the poses the robot has passed, move the first pose to the robot and the last to the new local goal, 1.5 m further along the global plan (teb_local_planner);
3. **optimise:** TEB's 4 passes of 5 steps (Section 5.3);
4. **drive** the band's first 0.1 s, and start the next cycle.

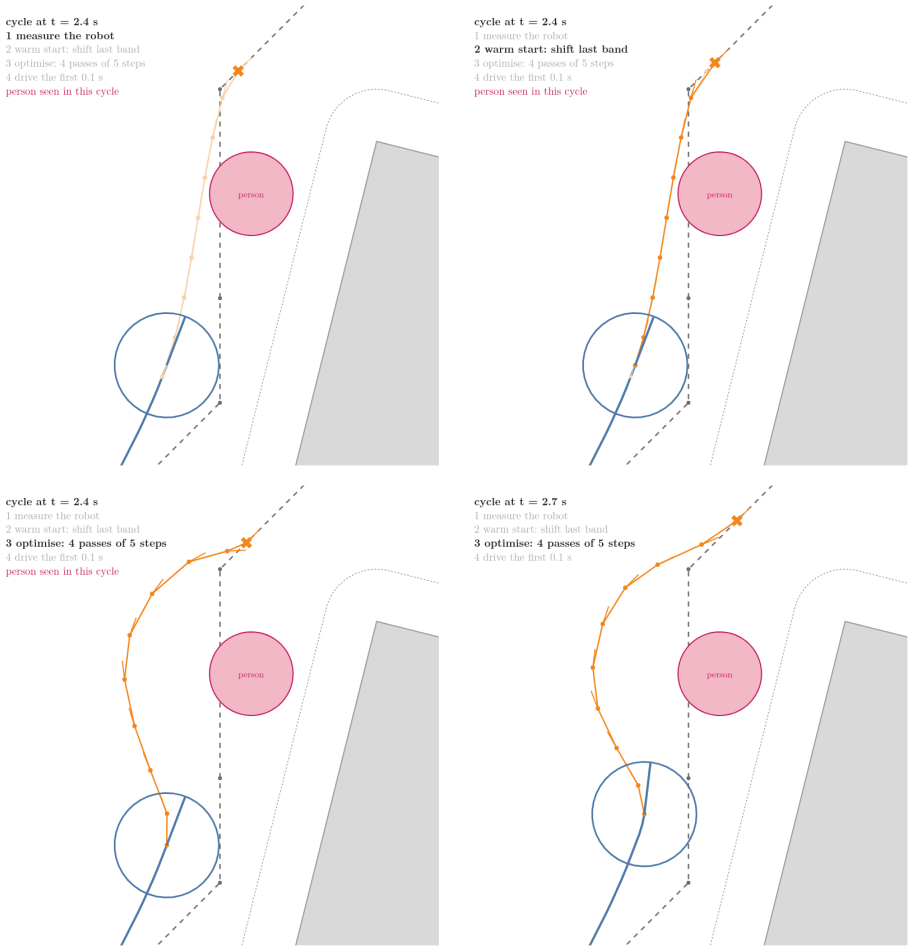


Figure 12: The receding-horizon loop, cycle by cycle, from t = 2.2 s to 2.7 s; the person steps onto the route at 2.4 s. Each cycle has three frames. 1 measure: the robot has driven 0.1 s along last cycle's band (pale). 2 warm start: the passed poses are dropped, the first pose is moved to the robot and the last to the new local goal (cross). 3 optimise: the new band (orange). Then the robot drives the band's first 0.1 s

In the cycle at 2.4 s the warm start runs straight through the person, because it was planned before they appeared; one solve bends it 0.146 m clear (Figure 8). The next cycles start from that bent band and only adjust it. Over the whole drive the loop runs 81 cycles; the band holds between 3 and 12 poses, about 0.29 s apart, and shrinks to 3 poses as the robot reaches the door (notebook).

Why solve again every cycle. One trajectory answers for one start (Section 3.2). Each cycle starts from where the robot really is, so errors are corrected: the band of the first cycle asked for 0.649 m/s^2, the robot gave 0.5 m/s^2, and the next cycle simply planned from the slower robot (Section 5.2). Each cycle also sees what the sensors see, so the person entered the plan in the cycle they appeared. TEB solves its problem "repeatedly at a rate faster than the robot control cycle" and feeds each answer back "to warm-start consecutive optimizations" (Rösmann et al. 2017). Solving an optimal control problem again at every step, from the

measured state, and applying only the first part, is also the core of model predictive control, which the control chapters teach (Rösmann et al. 2021).

The loop keeps the robot moving with no plan longer than 1.5 m. When there is no surprise to react to, one solve over the whole drive does better.

9. Refine once: one solve for a whole drive

> **Key point:** The same optimisation can run once, over a whole drive, started from a search's path: from the A* route, collocation finds the least-time drive to the door pose, 6.18 s, against 11.81 s for driving the route by stopping and turning at each corner. A controller then follows it.

The opening's route said where, not when; the local planner timed it 1.5 m at a time. With no person, one solve can time the whole route at once.

The drive. We take the A* route as the first guess, with the robot's position in place of the start cell's centre and the door pose in place of the goal cell's centre, and ask trapezoidal collocation (Section 6) for the least time. For this problem we use a solver that keeps every constraint exactly, SciPy's SLSQP, which solves a quadratic program at each step (Virtanen et al. 2020); methods of this kind find a locally best trajectory when the first guess is close to one (Lynch and Park §10.7). Figure 13 drives the answer against the route driven by stopping and turning at each corner.

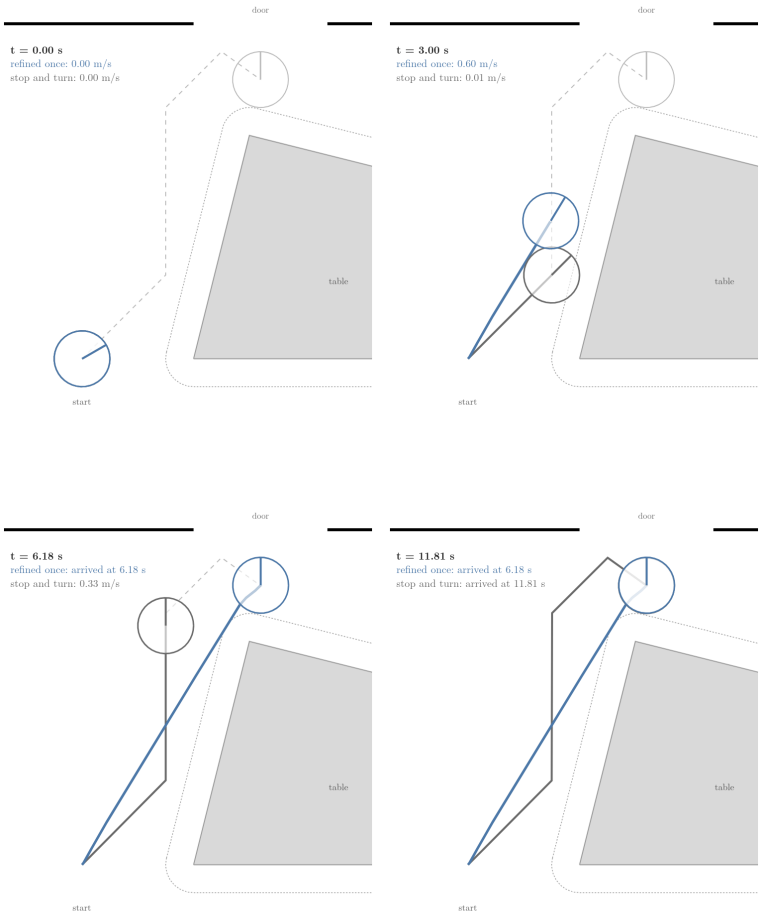


Figure 13: Refine once: two drives from rest at (2.0, 1.0) facing 30° to rest at the door pose (3.6, 3.5) facing 90°, one frame per 0.2 s. Grey: the A* route (pale dashed) driven by turning on the spot to face each next corner, driving to it from rest to rest, and turning to the door's heading at the end: 11.81 s. Blue: the least-time trajectory that collocation finds once, starting from that route (its controls replayed in small simulation steps): 6.18 s. The dotted band round the table is where the disk's centre may not go

The refined drive reaches the door pose in 6.18 s, against 11.81 s for stopping and turning along the route (notebook). Figure 14 shows its speeds and controls over time.

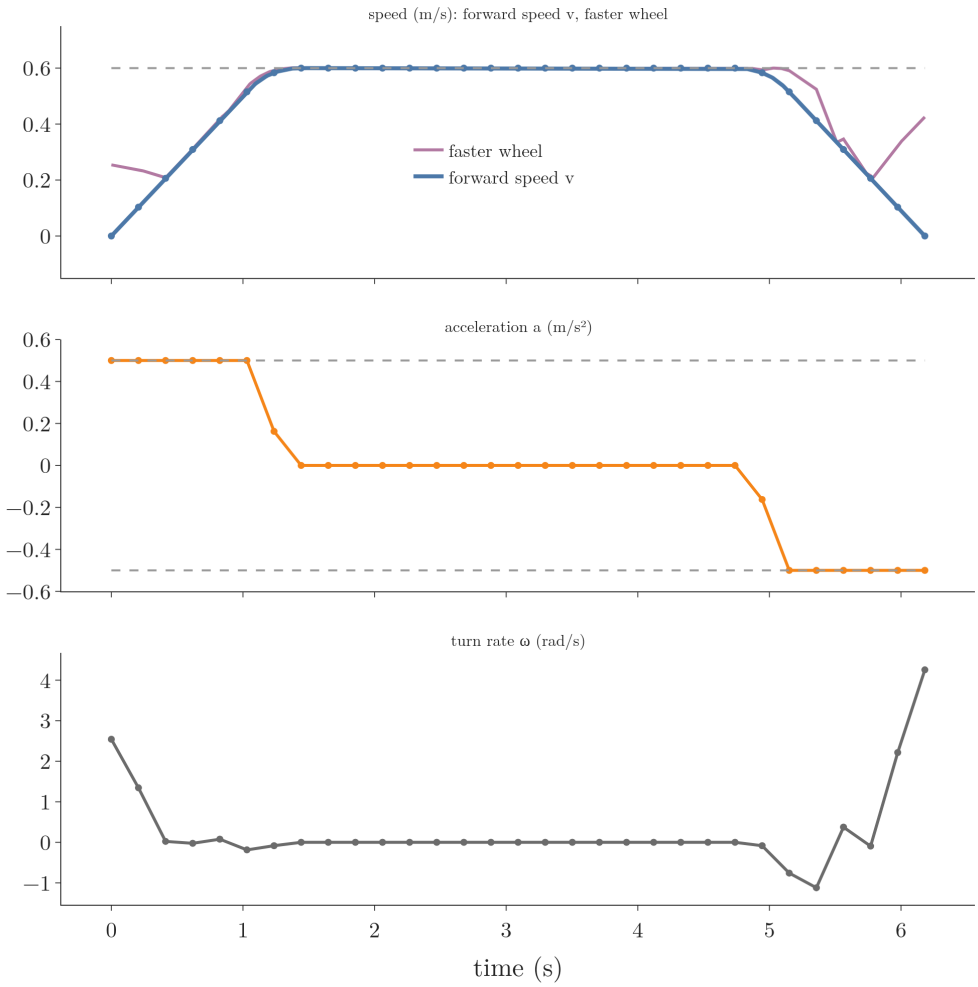


Figure 14: The least-time drive to the door pose found by collocation, 6.18 s, over time; lines: the controls replayed in small simulation steps; dots: the 31 knot points. Top: the forward speed v (blue) and the faster wheel (purple), both at or under the 0.6 m/s limit (dashed). Middle: the acceleration a: $+0.5$ m/s^2 to speed up, 0 while cruising, -0.5 m/s^2 to brake. Bottom: the turn rate ω: a fast turn at the start and at the end, where the speed is low and the wheels allow up to 6 rad/s

Every limit that can be used is used:

- the acceleration sits at $+0.5$ m/s^2 until the speed reaches 0.6 m/s, and at -0.5 m/s^2 to brake;
- the faster wheel sits at its 0.6 m/s limit while the robot cruises;
- the disk passes the table's corner with no room to spare;
- the turns happen at the two ends, where the speed is low and the wheels allow a fast turn.

A straight line from start to door pose, which ignores the table and every turn, could not be driven in less than 6.147 s; the refined drive needs only 0.034 s more (notebook). The local

planner of Section 8, which plans only 1.5 m at a time, took 6.7 s with nobody in the way (notebook).

Checking between the knot points. The constraints hold at the knot points and halfway between them, and nowhere else is checked. Replaying the found controls in small simulation steps from the start, the drive ends 0.003 m from the door pose and its speed never passes 0.6 m/s (notebook). Collocation enforces the motion rules only at its points, so its error must be checked between them (Kelly 2017, §5.3).

After the solve. The robot then follows the drive with a controller that measures where it is and corrects the commands, the subject of feedback control; Apollo parks this way, from a Hybrid A* path, and MoveIt moves arms this way, from a sampling planner's path (Section 10). The room now runs both loops of Section 2.3: re-planning every cycle round the person, and refining once over a whole drive. Real robots and cars run the same two loops at larger scale.

10. Real systems

> **Key point:** TEB, Autoware, Apollo and MoveIt can all optimise trajectories with the ideas of this Note: TEB as one sparse least-squares band per cycle, Autoware and Apollo as a chain of quadratic programs, path first and speed second, and MoveIt as a sampling planner's path refined once. Each pairs a search or a last answer with an optimiser.

The opening's room is small; real robots and cars meet the same three problems, a plan without a clock, things the map lacks, and little time, at larger scale.

10.1 Solvers, and path then speed

Three solvers do most of this work:

- **OSQP** (G-2926) solves quadratic programs, a squared cost with linear bounds, fast and warm-started from the last answer; Autoware's smoothers and Apollo's path and speed optimisers use it (Stellato et al. 2020; autoware_launch; Apollo planning);
- **g2o** (G-2927) solves sparse least-squares problems whose terms each join a few variables, with Levenberg–Marquardt steps; TEB uses it (Kümmerle et al. 2011; Rösmann et al. 2013);
- **IPOPT** (G-2928) solves general problems with curved costs and constraints; one of Apollo's two parking optimisers and the newer MPC local planner of TEB's authors use it (Wächter and Biegler 2006; Apollo planning; Rösmann et al. 2021).

A car's trajectory problem is often split in two: first the path, its shape, then the speed along it. Finding a path first and then the speed profile along that fixed path, so that each part is a simpler problem, is **path–speed decomposition** (G-2925) (Zhou et al. 2020; Fan et al. 2018). Autoware and Apollo both work this way. Speed profiles along a fixed path (RO-139) teaches the speed half, and on-road planning by optimisation (RO-229) the path half; both Notes come later.

10.2 TEB in ROS and Nav2

In ROS, TEB solves the band of Section 5 once per control cycle on up to 3 m of the global plan, warm-started from the last band, and checks the first 5 poses against the obstacle map before it sends a command (teb_local_planner). Figure 15 shows the cycle; the arrow back is the warm start. Classical local planning (RO-078) teaches TEB as a ROS and Nav2 controller.

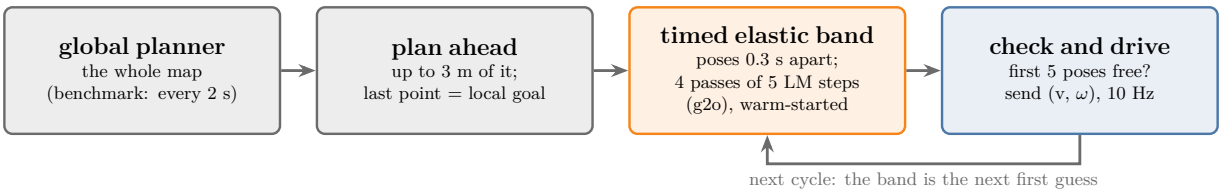


Figure 15: TEB in ROS and Nav2: the global planner plans the whole map (every 2 s in a ROS benchmark); up to 3 m of its plan is taken, the last point being the local goal; the timed elastic band, poses 0.3 s apart, is optimised with 4 passes of 5 Levenberg–Marquardt steps in g2o, warm-started; the first 5 poses are checked for collisions and the speed and turn rate (v, ω) are sent, 10 times a second in its tutorial; the band is the next cycle's first guess

10.3 Autoware

Autoware's lane driving runs three quadratic programs in a row, each solved with OSQP again at every new input (autoware_launch; Autoware path smoother; Autoware path optimizer; Autoware velocity smoother):

1. the elastic band of Section 4 smooths the first 100 m of the 300 m behaviour path (points 1 m apart, each moving sideways at most 0.1 m);
2. MPT, the model predictive trajectory, fits a bicycle model of the car along it, with the car covered by 3 circles that soft constraints keep inside the road;
3. the velocity smoother sets the speed over 200 m within acceleration and jerk limits.

Figure 16 shows the chain: the two orange steps fix the path, the blue one the speed. On-road planning by optimisation (RO-229) teaches it in full.

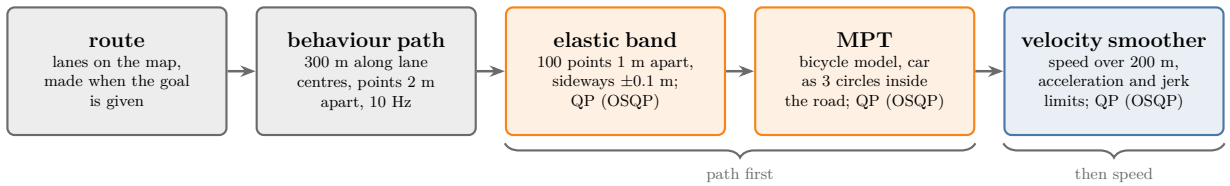


Figure 16: Autoware Universe lane driving: the route over the lanes of the map, made when the goal is given; the behaviour path, 300 m along lane centres with points 2 m apart, 10 times a second; the elastic band, 100 points 1 m apart moving sideways at most 0.1 m, a quadratic program (OSQP); MPT, a bicycle model with the car as 3 circles inside the road, a quadratic program (OSQP); the velocity smoother, the speed over 200 m within acceleration and jerk limits, a quadratic program (OSQP). The path comes first, then the speed

10.4 Apollo

Apollo's public-road planner builds an 8 s trajectory in three steps (Apollo planning; Zhang et al. 2020; Fan et al. 2018):

1. the reference line: the lane's centre line, smoothed with the same second-difference cost, in its own loop every 50 ms;
2. the path, a piecewise-jerk quadratic program (Section 6.3), every cycle;
3. the speed, a search over station and time followed by a piecewise-jerk quadratic program, every cycle.

For parking it refines once: a Hybrid A* path is smoothed by quadratic programs and timed by a speed quadratic program, each step taking 0.035 s on average in its authors' tests (Zhou et al. 2020). Figure 17 shows both chains: the public-road one re-plans every cycle, the parking one refines once. On-road planning by optimisation (RO-229) teaches both.

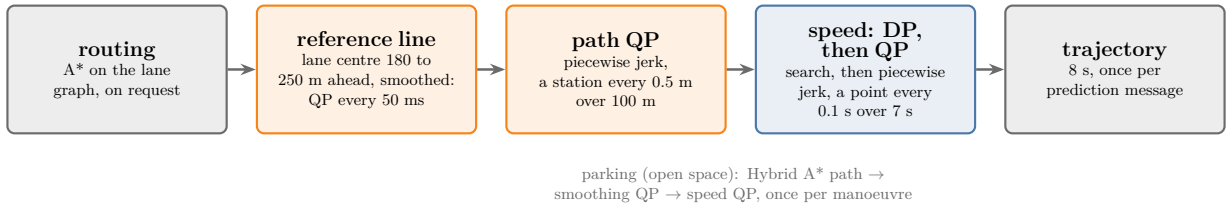


Figure 17: Apollo: routing, A* on the lane graph on request; the reference line, the lane centre 180 to 250 m ahead, smoothed by a quadratic program every 50 ms; the path, a piecewise-jerk quadratic program with a station every 0.5 m over 100 m; the speed, a search (dynamic programming) then a piecewise-jerk quadratic program with a point every 0.1 s over 7 s; an 8 s trajectory once per prediction message. Below: parking, a Hybrid A* path, then a smoothing quadratic program, then a speed quadratic program, once per manoeuvre

10.5 MoveIt

MoveIt can chain planners for one motion request: a sampling planner from OMPL (G-2814) finds a path free of collisions, CHOMP or STOMP optimises it with that path as the first guess, and a last step times the path within the joints' speed and acceleration limits (MoveIt 2 code; MoveIt 2 docs). Figure 18 shows the chain, the refine-once loop of Figure 3. Optimisation-based arm motion planning (RO-230) teaches CHOMP, STOMP and TrajOpt.

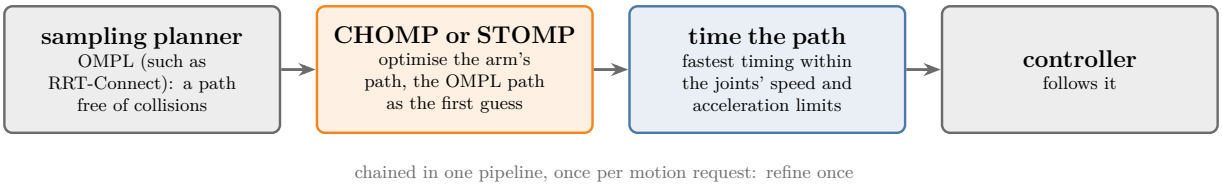


Figure 18: MoveIt 2: a sampling planner from OMPL (such as RRT-Connect) finds a path free of collisions; CHOMP or STOMP optimises the arm's path, with the OMPL path as the first guess; a last step gives the path its fastest timing within the joints' speed and acceleration limits; a controller follows it. Chained in one pipeline, all of it runs once per motion request: refine once

11. Summary

Idea	What it does	In the room
global plan and local planner	the map's route once; the next metres every cycle	3.414 m once; 1.5 m every 0.1 s
soft constraint	a penalty that grows with the excess	weight 2 lets the wheels reach 0.62 m/s
elastic band	second differences, sideways moves, bounds	cost 0.75 to 0.397 m^2
timed elastic band	poses with time steps, sparse least squares	band bent 0.146 m clear of the person
direct collocation	states and controls at knot points, the trapezoid rule between them	187 numbers for the least-time drive
warm start from a search	picks the side of an obstacle	cost 1.49 instead of 5.82
receding horizon	solve again every cycle from the last answer	door at 8.0 s with the person
refine once	one solve for a whole drive	6.18 s instead of 11.81 s

- The robot reaches the door at 8.0 s instead of the 6.7 s of a clear room and passes the person 0.14 m clear, because its local planner solves a trajectory optimisation problem over the next 1.5 m every 0.1 s, so the person entered the plan in the cycle they appeared.
- The **global planner** (G-2910) plans the whole map rarely and the **local planner** (G-2911) plans the next few metres often, because a whole-map plan is too slow to remake every cycle and knows neither time nor new obstacles; they meet at the **local goal** (G-2929).
- A **trajectory** (G-2886) gives the state at every moment, so it says when, not only where; **trajectory optimisation** (G-2887) chooses its **decision variables** (G-2915) for the least cost within its constraints, because one good trajectory is affordable where a rule for every state is not.
- A **hard constraint** (G-2916) must hold exactly and a **soft constraint** (G-2917) is a penalty, so a soft limit can be broken when breaking it pays: weight 2 lets a band's wheels reach 0.62 m/s, weight 1000 holds them to 0.550 m/s.
- An **elastic band** (G-2919) smooths a route by minimising squared **second differences** (G-2918) within bounds, so corners pull in by up to 0.1 m and the cost falls from 0.75 to 0.397 m^2; the 1, 5, 6 matrix is the same in Autoware and Apollo.
- A **timed elastic band** (G-2920) adds the time between poses, so speeds and accelerations come from differences between neighbours, and the **Levenberg–Marquardt method** (G-2921) solves it fast because its matrix is mostly zeros (47 percent filled for 10 poses, 9.7 percent for 47).
- **Transcription** (G-2888) turns a trajectory into a list of numbers; **direct collocation** (G-2893) keeps the state and the controls at **knot points** (G-2895) and ties neighbours by **collocation constraints** (G-2894), the trapezoid rule, so nothing is simulated and

every limit is checked at a point: the least-time drive is 187 numbers, and the rule gives the first 0.6364 s of the straight drive exactly, 0.1013 m.

- TEB is **finite-difference collocation** (G-2923) and Apollo's **piecewise-jerk optimisation** (G-2924) is collocation on a chain of integrators, because both write the motion rules as equations between neighbouring points.
- An optimiser stops in the **local minimum** (G-1110) of the valley it starts in and never leaves its **homotopy class** (G-2922), so a search must choose the side: with the person 0.25 m further left the warm start scraped through at cost 5.82, a search start passed at cost 1.49.
- A **warm start** (G-2914) on a **receding horizon** (G-2913) makes each cycle a correction of the last answer from where the robot really is, so the loop takes in new obstacles and the robot's errors every 0.1 s; refined once from the A* route, the whole drive takes 6.18 s against 11.81 s for stopping and turning.
- Real systems solve these problems with **OSQP** (G-2926), **g2o** (G-2927) or **IPOPT** (G-2928), and cars plan by **path–speed decomposition** (G-2925), because two small problems solve faster than one large one.

The robot can now plan, every 0.1 s, a timed trajectory round things the map does not have, or refine a whole drive once. Still missing: whether collocation suits a 0.1 s cycle better than shooting, the other way to turn the problem into numbers, and how a solver can take better steps from the robot's model, iLQR: the next Note (RO-231). Beyond that, a real robot slips and drifts away from its plan between plans; following a trajectory needs feedback, which measures where the robot is and corrects the commands, the subject of the next chapter, feedback control.

12. Sources

Built from

- Autonomy Talks, "Sampling-Based Motion Planning and Direct Optimal Control", YouTube, https://www.youtube.com/watch?v=LWBjOTBZvZ4
- Matthew Kelly, "Introduction to Trajectory Optimization", a talk with slides given at Cornell University, YouTube, https://www.youtube.com/watch?v=wlkRYMVUZTs
- Paden, B., Čáp, M., Yong, S. Z., Yershov, D. and Frazzoli, E. (2016). "A survey of motion planning and control techniques for self-driving urban vehicles". *IEEE Transactions on Intelligent Vehicles* 1(1), 33–55, arXiv 1604.07446: §II (the decision-making hierarchy), §IV-A (variational, graph-search and incremental methods; a coarse search as the initial guess), §IV-C (penalty methods; direct and indirect methods) (Paden et al. 2016)
- Rösmann, C., Feiten, W., Wösch, T., Hoffmann, F. and Bertram, T. (2013). "Efficient trajectory optimization using a sparse model". *European Conference on Mobile Robots (ECMR)*, free PDF (Rösmann et al. 2013)
- Rösmann, C., Hoffmann, F. and Bertram, T. (2015). "Planning of multiple robot trajectories in distinctive topologies". *ECMR*, free PDF (Rösmann et al. 2015)
- Rösmann, C., Hoffmann, F. and Bertram, T. (2017). "Kinodynamic trajectory optimization and control for car-like robots". *IEEE/RSJ IROS*, §I–II, free PDF (Rösmann

et al. 2017)
- Rösmann, C., Makarow, A. and Bertram, T. (2021). "Online motion planning based on nonlinear model predictive control with non-Euclidean rotation groups". *European Control Conference*, arXiv 2006.03534: §I (TEB as finite-difference collocation; DWA as a receding-horizon scheme), §IV-B (benchmark: global plan every 2 s, local planners at 10 Hz, 1.5 m lookahead) (Rösmann et al. 2021)
- teb_local_planner, GitHub, https://github.com/rst-tu-dortmund/teb_local_planner (noetic-devel): `cfg/TebLocalPlannerReconfigure.cfg` (defaults), `src/optimal_planner.cpp` (passes, weight doubling), `src/timed_elastic_band.cpp` (`autoResize`, `updateAndPruneTEB`), `include/teb_local_planner/g2o_types/` (edges and penalties); ros2-master README (Nav2 port) (teb_local_planner)
- teb_local_planner tutorials and FAQ, ROS wiki (Internet Archive copies), http://wiki.ros.org/teb_local_planner/Tutorials (TEB FAQ)
- Kelly, M. (2017). "An introduction to trajectory optimization: how to do your own direct collocation". *SIAM Review* 59(4), 849–904, https://doi.org/10.1137/16M1062569, free PDF from the author (Kelly 2017)
- Tedrake, R. *Underactuated Robotics*, ch. 10 "Trajectory optimization", https://underactuated.mit.edu/t (Tedrake, Underactuated ch. 10)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §10.7 "Nonlinear optimization", http://modernrobotics.org (Lynch and Park §10.7)
- Autoware Universe, GitHub: `planning/autoware_path_smoother` (README, `docs/eb.md`, `src/elastic_band.cpp`), https://github.com/autowarefoundation/autoware_universe (Autoware path smoother); `planning/autoware_path_optimizer` (README, `docs/mpt.md`) (Autoware path optimizer); autoware_core `planning/autoware_velocity_smoother` and `planning/autoware_mission_planner`, https://github.com/autowarefoundation/autoware_core (Autoware velocity smoother; Autoware mission planner); autoware_launch `autoware_planning_config` parameter files, https://github.com/autowarefoundation/autoware_laun (autoware_launch)
- Apollo, GitHub, https://github.com/ApolloAuto/apollo (master): `modules/planning` (`planning_base/gflags/planning_gflags.cc`, `scenarios/lane_follow/conf/pipeline.pb.txt`, `tasks/piecewise_jerk_speed`, `tasks/open_space_trajectory_provider`, `planning_base/math/` `modules/routing`, `docs/08_Planning/Open_Space_Planner.md` (Apollo planning)
- Fan, H. et al. (2018). "Baidu Apollo EM motion planner". arXiv 1807.08048 (Fan et al. 2018)
- Zhang, Y. et al. (2020). "Optimal vehicle path planning using quadratic optimization for Baidu Apollo open platform". *IEEE Intelligent Vehicles Symposium*, arXiv 2112.02132 (Zhang et al. 2020)
- Zhou, J. et al. (2020). "DL-IAPS and PJSO: a path/speed decoupled trajectory optimization and its application in autonomous driving". arXiv 2009.11135 (Zhou et al. 2020)
- MoveIt 2, GitHub, https://github.com/moveit/moveit2 (main): `moveit_ros/planning/planning_pi` `moveit_planners/chomp`, `moveit_planners/stomp` (MoveIt 2 code); MoveIt 2 how-to guides "Using CHOMP Planner" and "STOMP Motion Planner", https://moveit.picknik.ai/main/doc/how_to_guides/chomp_planner/chomp_planner_tutorial.html

(MoveIt 2 docs)

Other references

- ROS navigation stack, DWA local planner, its configuration file DWAPlanner.cfg (default settings), GitHub, https://github.com/ros-planning/navigation (DWAPlanner.cfg)
- Nocedal, J. and Wright, S. J. (2006). *Numerical Optimization*, 2nd ed. Springer. §10.3 "Algorithms for nonlinear least-squares problems" (Gauss–Newton and Levenberg–Marquardt) (Nocedal and Wright §10.3)
- Stellato, B., Banjac, G., Goulart, P., Bemporad, A. and Boyd, S. (2020). "OSQP: an operator splitting solver for quadratic programs". *Mathematical Programming Computation* 12, 637–672 (Stellato et al. 2020)
- Kümmerle, R., Grisetti, G., Strasdat, H., Konolige, K. and Burgard, W. (2011). "g2o: a general framework for graph optimization". *IEEE ICRA* (Kümmerle et al. 2011)
- Wächter, A. and Biegler, L. T. (2006). "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming". *Mathematical Programming* 106, 25–57 (Wächter and Biegler 2006)
- Virtanen, P. et al. (2020). "SciPy 1.0: fundamental algorithms for scientific computing in Python". *Nature Methods* 17, 261–272: the solvers `lsq_linear` and `minimize(method="SLSQP")` used in the notebook (Virtanen et al. 2020)
- Tedrake, R. *Robotic Manipulation*, ch. "Motion planning", https://manipulation.mit.edu/trajectories.ht kinematic trajectory optimisation, local minima, and the sampling planner's path as the initial guess
- Northwestern Robotics, "Modern Robotics, Chapter 10.7: Nonlinear Optimization", YouTube, https://www.youtube.com/watch?v=-z6Rlei8iPE: the video version of Lynch and Park §10.7

13. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Global planner (G-2910)	The planner that finds a route over the whole map, from the robot to its goal, and runs rarely, such as A* on a room's grid or a road network; its route, the global plan, guides the local planner but has no timing and knows only the map.
Behavioural layer (behavioural decision making) (G-2912)	The level of planning between route planning and motion planning that decides which task the vehicle carries out now, such as following the lane, stopping at a line or giving way, from the route and what it perceives; the motion planner then plans that task.

Term	Meaning
Local planner (G-2911)	The planner that turns the next few metres or seconds of the global plan into a timed trajectory within the robot's limits and around obstacles its sensors see, solving again many times a second, such as TEB at 10 Hz over 1.5 m; it handles what the global plan cannot.
Local goal (virtual goal) (G-2929)	The pose on the global plan at the end of the local planner's window, facing along the plan, which the local planner plans to reach; it moves along the plan with the robot, and becomes the real goal when that comes within reach.
Warm start (G-2914)	Starting a solver from an earlier answer, such as the last cycle's trajectory or a planner's path, instead of from scratch; each solve then only corrects a nearby answer, and the starting point also decides which local minimum it reaches.
Receding horizon (G-2913)	Planning over a window of fixed length that starts at the robot's current state, applying only the first part of the plan, then planning again from the new state, so the window slides forward with the robot; it lets a planner take in new obstacles and correct errors every cycle.
Trajectory (G-2886)	The state of a robot at every moment of a time interval, together with the controls that produce it, such as a local planner's band of poses with times; unlike a path, it says when the robot is where and how fast it moves.
Decision variables (G-2915)	The numbers an optimisation is free to choose, such as the poses and time steps of a band or the controls at each interval; the solver changes them to lower the cost while the constraints hold.
Trajectory optimisation (G-2887)	Choosing the numbers that describe a trajectory to make a cost such as the time or the effort as small as possible while the motion rules, the start and the end, the limits and the clearance from obstacles hold; it computes one good trajectory where a rule for every state would be far too large.

Term	Meaning
Hard constraint (G-2916)	A constraint the answer must meet exactly, so no answer that breaks it is accepted, such as a wheel speed that may never pass 0.6 m/s; it holds whatever it costs.
Soft constraint (penalty) (G-2917)	A constraint replaced by an extra cost that is 0 while it holds and grows with how far it is broken, such as a weighted squared excess over a speed limit; the problem then has no constraints and is easy to solve, but the answer may break the limit a little when that lowers the total cost.
Second difference (G-2918)	For three neighbouring points of a path, the two outer points minus twice the middle one, $p_{k-1} - 2p_k + p_{k+1}$; it is zero on a straight, evenly spaced stretch and large where the path bends, so its square measures how unsmooth the path is.
Elastic band (G-2919)	A path treated as a chain of points that a cost pulls straight, like a stretched rubber band, while bounds or obstacles hold it in place, and optimised; Autoware's smoother of this kind minimises squared second differences with small sideways moves as a quadratic program.
Timed elastic band (TEB) (G-2920)	A trajectory written as a list of poses with the time between neighbours, optimised for the least total time while penalties keep the motion rule, the speed and acceleration limits and the clearance from obstacles; a ROS and Nav2 local planner that solves it again every cycle from its last answer.
Levenberg–Marquardt method (G-2921)	A solver for sums of squares that takes Gauss–Newton steps, jumping to the lowest point of a bowl built from straight-line guesses of each term, blended with a small step downhill that grows when a step fails and shrinks when it succeeds; it converges fast near the answer and safely far from it.

Term	Meaning
Transcription (G-2888)	Turning a trajectory optimisation problem, whose unknowns are functions of time, into one whose unknowns are a finite list of numbers, such as a band's 10 poses and the 9 time steps between them, 39 numbers, so that an optimiser can search over them.
Direct collocation (G-2893)	Transcription in which the states and controls at a row of points in time are all unknowns and the motion rules are equations between neighbouring points; nothing is simulated, so limits along the way are easy to add.
Knot point (collocation point) (G-2895)	One of the points in time at which collocation keeps the state and controls as unknowns, such as 31 points 0.206 s apart over a 6.18 s drive; more knot points make the solution more accurate.
Collocation constraint (trapezoidal) (G-2894)	The equation that ties neighbouring knot points through the motion rules; in trapezoidal collocation the change in state equals the time step times the average of the rates at the two points, $x_{k+1} - x_k = \frac{h}{2}(f_k + f_{k+1})$, so a simulation is not needed.
Finite-difference collocation (G-2923)	Direct collocation in which the motion rules are written as differences between neighbouring points, such as a speed equal to the distance between two poses over the time between them; TEB is derived this way.
Piecewise-jerk optimisation (G-2924)	A quadratic program that keeps a quantity, its slope and its second slope at stations as unknowns and holds the third slope (the jerk) constant between neighbours, so neighbours are tied by exact equations; Apollo plans a car's sideways path and its speed this way.
Homotopy class (G-2922)	A set of paths with the same start and goal that can be bent into one another without crossing an obstacle, such as all paths passing a person on the left; an optimiser never leaves the class it starts in, so a search must pick the class.

Term	Meaning
OSQP (G-2926)	A solver for quadratic programs, a squared cost with linear bounds, that can start from its last answer; Autoware's smoothers and Apollo's path and speed optimisers use it every planning cycle.
g2o (G-2927)	A library that solves sparse least-squares problems written as a graph, each term joining a few variables, with Gauss–Newton or Levenberg–Marquardt steps; TEB solves its band with it.
IPOPT (G-2928)	A solver for general optimisation problems with curved costs and constraints; one of Apollo's parking optimisers and some MPC planners use it.
Path–speed decomposition (path then speed) (G-2925)	Planning a trajectory in two steps, first the shape of the path and then the speed profile along that fixed path, so each step is a smaller, simpler problem; Autoware and Apollo plan this way.
Route planning (mission planning) (G-2870)	Finding the best way from where a vehicle is to its destination on a map of the road network, by travel time or by distance, such as the 492.8 s route across the Chicago map; the top level of planning, which picks the roads and lanes and leaves the motion to the levels below.
State x_t (G-2314)	The quantities a robot keeps track of at time t because they decide what happens next, such as its pose, its speeds and moving things around it; it is what localization and filtering estimate.
Control input u (G-2298)	The numbers we choose to drive a robot, such as $u = (v, \omega)$; the model turns them into motion.
Kinematic model (G-2297)	Equations that give how a robot's configuration changes from its speed inputs alone, ignoring masses and forces, such as $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$; used to predict motion at low speed.
Constraint (G-456)	A condition the solution must satisfy; in SVM, $y_i(w^T x_i + b) \geq 1$ for every training point.
Policy (G-1512)	The agent's rules for which action to take.

Term	Meaning
Path smoothing (G-2842)	Moving the points of a found path, sweep after sweep, a little towards their found places and a little towards their neighbours, $y_i \leftarrow y_i + \alpha(x_i - y_i) + \beta(y_{i+1} + y_{i-1} - 2y_i)$; it rounds corners and removes needless steering, but on its own it knows nothing of obstacles, the turning circle or the goal heading.
Unit vector (G-2048)	A vector of length 1, used to describe a direction.
Dot product (G-634)	Multiplying two vectors entry by entry and adding the products, $u^{\mathsf{T}}x$, which gives one number. It is large when the vectors point the same way, so it measures similarity and gives projections.
Matrix (G-1180)	A table of numbers in rows and columns (a 2D tensor); it can hold a dataset (one row per observation), and multiplying a vector by it transforms the vector.
Quadratic program (G-1598)	A problem of finding the lowest point of a bowl-shaped function with squared terms (a convex quadratic) while meeting limits written as linear inequalities (constraints).
Nonholonomic constraint (G-2302)	A rule on velocities that does not follow from any equation on the pose, such as a wheel that cannot slide sideways; it limits which way a robot can move at each moment but not which poses it can reach.
Ordinary least squares (OLS, least squares) (G-1406)	The way linear regression finds its line with one formula (a closed-form method): the line with the smallest sum of squared errors.
Jacobian (G-980)	The table (matrix) of all first partial derivatives of a function with several inputs and outputs, one row per output and one column per input, $J_{ij} = \partial f_i / \partial x_j$; it shows how every output changes with every input near a point.
Sparse (G-1846)	Mostly made of zeros, such as a feature map whose values are almost all 0, or a model whose coefficients are mostly exactly 0.
Local minimum (G-1110)	A point lower than everything around it, but not the lowest overall.

Term	Meaning
OMPL (Open Motion Planning Library) (G-2814)	A library of sampling-based planners, such as PRM and RRT, that by design has no collision checker or robot model, so it can be joined to any; MoveIt uses it by default.